

Leveraging Large Language Models for Text Annotation

Diploma Thesis

Supervisor:

doc. Ing. František Darena, Ph.D.

Gabriel Addo

Brno 2024

Acknowledgement

I would like to take this opportunity to thank the supervisor of my thesis, doc. Ing. František Darena, Ph.D for insightful feedback, thoughtful critiques, and constructive suggestions that have challenged me to think critically and pushed me to produce the best i can. I am truly grateful for the time and effort you have invested in giving me guidance.

Statutory declaration

I hereby declare that this thesis entitled: Error! Use the Home tab to apply Název práce to the text that you want to appear here. was written and completed by me. I also declare that all the sources and information used to complete the thesis are included in the list of references. I agree that the thesis could be made public in accordance with Article 47b of Act No. 111/1998 Coll., Higher Education Institutions and on Amendments and Supplements to Some Other Acts (the Higher Education Act), and in accordance with the current Directive on publishing of the final thesis.

I am aware that my thesis is written in accordance with Act. 121/2000 Coll., on Copyright and therefore Mendel University in Brno has the right to conclude licence agreements on the utilization of the thesis as a schoolwork in accordance with Article 60(1) of the Copyright Act.

Before concluding a licence agreement on utilization of the work by another person, I will request a written statement from the university that the licence agreement is not in contradiction to legitimate interests of the university, and I will also pay a prospective fee to cover the cost incurred in creating the work to the full amount of such costs.

In Brno on 5th December, 2024

Keywords

Vue.js, velký jazykový model, rozpoznávání pojmenovaných entit, analýza sentimentu, NLP, Python, Django, Redis, REST, API, GPT, Langchain, tokenizace.

Contents

1	Introduction	11
1.1	Background	11
1.2	Problem statement.....	12
1.3	Research questions	13
1.4	Objectives of the study.....	14
1.5	Significance of the study	14
1.6	Organization of the paper.....	14
2	Literature review	16
2.1	Text annotation	16
2.1.1	Tasks in text annotation.....	16
2.1.2	Annotation methods.....	19
2.2	Large language models.....	20
2.2.1	Transformer-based LLM.....	20
2.3	BERT architecture	23
2.3.1	Training BERT.....	23
2.3.2	TinyBERT architecture.....	24
2.4	GPT architecture	24
2.5	Prompt engineering.....	25
2.6	Review of text annotation applications	25
2.7	Evaluation metrics.....	30
3	Methodology	32
3.1	Requirements gathering	32
3.2	User interface design.....	33
3.3	System design.....	34

3.4	Choice of technologies	34
3.4.1	Choice of technologies	34
3.4.2	Backend	35
3.4.3	Database	35
3.5	Model selection	35
3.6	Implementation	37
3.6.1	Frontend development	37
3.6.2	Backend integration	38
3.6.3	Large language model integration	38
3.6.4	Database implementation	38
3.7	Evaluation	38
4	Results	41
4.1	Requirements analysis	41
4.1.1	Functional requirements	41
4.1.2	Non-functional requirements	43
4.1.3	Use case diagram	44
4.2	User interface design	46
4.3	System design	48
4.4	Implementation	51
4.4.1	Database connection	51
4.4.2	Registration	52
4.4.3	Login	53
4.4.4	Project creation	54
4.4.5	Label creation	55
4.4.6	Import dataset	55
4.4.7	Adding annotators	57
4.4.8	Annotator comments	58
4.4.9	Finetuning BERT for named entity recognition	59

4.4.10	Finetuning TinyBERT for sentiment analysis	66
4.4.11	Annotation.....	70
4.4.11.1	Manual annotation	70
4.4.11.2	Manual annotation for named entity recognition	70
4.4.11.3	Manual annotation for sentiment analysis.....	71
4.4.11.4	Semi-automated annotation with LLMs	72
4.4.11.4.1	Named entity recognition with GPT-4	73
4.4.11.5	Sentiment analysis with GPT-4.....	76
4.5	Testing.....	78
4.5.1	Sentiment analysis with GPT-4 and BERT.....	79
4.5.2	Named entity recognition with GPT-4 and TinyBERT.....	82
5	Discussion	84
5.1	Limitations of the application.....	85
6	Conclusion	87
7	References	88
8	Abbreviations	96
9	List of figures	97
11	List of tables	98

1 Introduction

1.1 Background

The innovative process of developing Large Language models proved to be not just a productive tool in advancing the field of artificial intelligence and data annotation, but indeed a breakthrough in natural language processing. The explosion of LLMs has fundamentally changed how we think about language understanding and text generation (Wang et al., 2024) but it has also changed the way we think about the efficiency and accuracy of data annotation. Indeed, LLMs undoubtedly have the reasoning capacity and complexity to be referred to as revolutionary.

More broadly, the development of LLMs such as OpenAI's GPT series (OpenAI, 2023), Google's BERT (Devlin et al., 2018), Meta's LLaMA (Touvron et al., 2023) among others is an incredible AI accomplishment, constructed on the latest advances in deep learning and neural networks. Since they are trained on giant collections of text drawn from the internet, they can seize a big array with nuances, styles, and contexts of human language (Devlin et al., 2018). In addition, their scales allow them to achieve a high level of precision on the growing set of NLP tasks, from forming the most complicated of queries to understanding them to producing accurate and related text responses (Bharathi Mohan, Prasanna Kumar, Vishal Krishh, et al., 2024). This is not solely a technical progress but also a conceptual and theoretical one, implying that on the scale of data, machines are adequate to approach an extensive part of human linguistics abilities.

The growth of large language models (LLMs) has markedly improved the efficacy of fundamental Natural Language Processing (NLP) tasks, including Named Entity Recognition (NER) and sentiment analysis. Large Language Models like as BERT (Devlin et al., 2019) and GPT-4 (OpenAI, 2023) undergo pre-training on extensive corpora encompassing several languages, contexts, and domains, which facilitates their profound comprehension of syntax, semantics, and pragmatics. This linguistic proficiency enables LLMs to excel in entity recognition across diverse textual contexts, hence enhancing the precision of NER tasks (Wang et al., 2024). Moreover, LLMs have demonstrated exceptional efficacy in sentiment analysis by discerning subtle emotional indicators and contextual intricacies, hence enhancing their use in contemporary text annotation systems (Zhang et al., 2023). These developments highlight the revolutionary impact of LLMs in diminishing

manual labour, enhancing annotation quality, and facilitating scalable solutions for NLP applications.

The choice of Named Entity Recognition (NER) and sentiment analysis as primary tasks in a text annotation application is warranted by their essential functions in information extraction and the structuring of extensive textual material. Named Entity Recognition (NER), an essential component of natural language processing, facilitates the detection and categorisation of named entities in unstructured text. NER identifies entities such as individuals, locations, and organisations, converting text into structured data, so enabling subsequent applications that rely on organised, accessible information. This capability is crucial across multiple areas, including biomedical, financial, and legal sectors, where precise extraction of specific entities is vital for further analysis and decision-making (Keraghel et al., 2024). The use of Named Entity Recognition (NER) in a text annotation tool is essential, allowing users to effectively extract and classify information within the annotated corpus.

Sentiment analysis plays a critical role in understanding and interpreting subjective opinions expressed in textual data by categorizing text into sentiment-based classes such as positive, negative, or neutral. This capability is essential across various domains, including customer service, market analysis, and social media monitoring, where user opinions and emotions drive decision-making processes. Sentiment analysis enables organizations to gain actionable insights into customer feedback, brand perception, and emerging trends, all of which are crucial for strategic planning. By semi-automating sentiment categorization, applications can streamline the process of analysing large volumes of unstructured data, providing users with valuable summaries of sentiment distributions, and reducing the burden of manual interpretation (Pang & Lee, 2008).

1.2 Problem statement

The traditional text annotation processes, especially for use-cases such as Named Entity Recognition and sentiment analysis, are manual, time-consuming, inconsistent, and thus result in a bottleneck for data prepping and model building. The issues are exacerbated by the exploding volume and variety of data and the subtle differences in language usage and meaning in different areas and contexts (Ratner et al., 2020). Although the recent development of Large Language Models or LLMs to some extent leads to lessening of the burden of manual text annotation via automatization, there seems to be room for improvement: the high cost of

human-like accuracy and reliability of annotations, the challenge to scale the models for extensive data-sets, the risk of biases via the same model that is learned (Brown et al., 2020; Bender et al., 2021). Moreover, there is a problem of incorporating an LLM model into the existing text annotation workflow: it can be time-consuming, error-prone, costly or, vice versa, monotonous.

While recent advances in text annotation applications have demonstrated potential in enhancing the text annotation processes, there remains improvements that can be made. These include enhancing collaboration among annotators, reviewers, and project managers through real-time interaction, incorporating large language models to assist with automated or semi-automated annotation tasks (Devlin et al., 2019; Radford et al., 2019), and the development of a user-friendly text annotation application that empowers researchers and developers to fine-tune large language models efficiently.

Thus, this thesis seeks to address the overarching question: How can Large Language Models be leveraged to revolutionize the text annotation process for Named Entity Recognition and sentiment analysis tasks, thereby overcoming the limitations of manual annotation methods while ensuring real-time collaboration and better UX.

1.3 Research questions

The following are some of the questions the study wants to address.

1. What current applications are available that support text annotation, and how do they integrate with Named Entity Recognition (NER) and sentiment analysis tasks?
2. What are the main limitations and challenges faced by current text annotation applications, particularly in the context of Named Entity Recognition and sentiment analysis?
3. What improvements can be made to existing text annotation applications to better serve the tasks of Named Entity Recognition and sentiment analysis?
4. Which advanced technologies, particularly Large Language Models, could be adopted or further developed to enhance text annotation tools for NER and sentiment analysis?

1.4 Objectives of the study

To achieve the objectives of this research, the thesis is organized around the objective which is to explore large language models (LLMs) for developing advanced text annotation applications. The primary aim of this thesis is to develop a user-friendly text annotation application that leverages LLMs to facilitate collaboration among annotators, ensure high-quality annotations, and reduce manual workload through semi-automation, specifically in tasks like Named Entity Recognition (NER) and sentiment analysis.

1.5 Significance of the study

The significance of the thesis focused on "Leveraging Large Language Models for Text Annotation," particularly in the context of Named Entity Recognition (NER) and sentiment analysis, spans several dimensions of both academic and practical importance. This research holds the potential to contribute significantly to the field of natural language processing (NLP) and artificial intelligence (AI) by addressing critical challenges and unlocking new possibilities.

1.6 Organization of the paper

The thesis is structured into six major sections. The Introduction establishes the context by outlining the background, problem statement, research aims, and significance of the study. The Literature Review examines current research on text annotation, large language models, pertinent technologies and evaluation metrics, providing a basis for the methodology and execution. The Methodology section establishes the system design, model selection used in the development of the text annotation application, along with graphic diagrams of the system architecture.

The methodology section also defines the technical development of the program, addressing elements such as user registration, project configuration, dataset importation, and model optimisation. This section contains code samples and pictures of the user interface. The Results section assesses testing of the application, the efficacy of large language models in sentiment analysis and named entity identification tasks, utilising quantitative indicators to highlight the strengths and limitations of each model.

The discussion analyses these findings, acknowledges limitations like computational requirements and model hallucinations. The conclusion

encapsulates the study's contributions and consequences. The thesis concludes with a thorough References section, cataloguing all sources utilised during the research.

2 Literature review

Large Language Models have been extensively utilised in Natural Language Processing because of their capacity to store semantic and syntactic information from extensive corpora. Devlin et al. (2019) emphasised the transformative influence of bidirectional models, which incorporate both left and right context, on language comprehension in their foundational study on BERT (Bidirectional Encoder Representations from Transformers). Since then, GPT models have further advanced the field, with applications ranging from text generation to language understanding, thus revolutionizing automated text annotation (Brown et al., 2020). This section provides a detailed exploration of text annotation, focusing on its fundamental tasks and methods, followed by an analysis of large language models that have been pivotal in advancing this field.

2.1 Text annotation

Text annotation is the process of incorporating interpretative linguistic information into an electronic corpus of written language data. “Annotation” may also denote the outcome of this process (Leech, 1997). This process is essential for the training of machine learning models in natural language processing tasks (NLP). Traditional text annotation tools have been developed over the years to support manual processes of annotation in a corpus and these tools offer some features that greatly speed up the process of annotation.

In the field of data science and natural language processing (NLP), text annotation demands an in-depth understanding of the problem, the domain, and the dataset. The process involves marking important and relevant features within a corpus. This process can also be done during the labeling of images in image classification.

2.1.1 Tasks in text annotation

As the cornerstone of NLP model training, text annotation includes a variety of tasks, each tailored to capture different dimensions of language use and interpretation. The evolution of annotation methodologies has been pivotal in propelling the field of NLP forward, allowing for increasingly sophisticated interactions between computers and human language. This literature review delves into the diverse landscape of text annotation which then dives deep into exploring its primary types:

1. Entity recognition: Entity Recognition focuses on assigning labels to key phrases, named entities or parts of speech in a corpus. This step also involves identifying and categorizing entities such as persons, locations, organizations, dates, times etc. A breakdown of entity recognition can be captured below:

- i. Named Entity Recognition (NER) – Named Entity Recognition is a subset of entity recognition in which entities are identified and classified into categories from unstructured text. NER makes classification on word or even at the character level, giving each word or a character a category or a label (Keraghel, Morbieu, and Nadif, 2024).

In the sentence "The movie was directed by Christopher Nolan and starred Leonardo DiCaprio," we can label the named entities as follows:
NER Labels:

"The movie was directed by [PERSON] [Christopher Nolan] and starred [PERSON] [Leonardo DiCaprio]."

The "[PERSON]" label signifies that the identified entities are names of people. In NER, other common labels include "[ORG]" for organizations, "[LOC]" for locations, "[DATE]" for dates, and so on. The purpose of NER is to extract and categorize these named entities to enhance the understanding of the text and support various natural language processing tasks.

Manually marking or highlighting entities like "Christopher Nolan," and "Leonardo DiCaprio" helps train a model to recognize and categorize similar entities in new texts.

This is useful for applications like extracting information about movies and their cast.

- ii. Key phrase tagging – Key phrase tagging is a process of extracting key phrases from text that represent the content's main themes or ideas. This task is mainly used for summarizing, indexing, facilitating information retrieval, and enhancing user search experience (Song et al., 2023).

In the sentence "Climate change represents one of the most significant global challenges of our time. Rising average temperatures, extreme weather events, and declining biodiversity are just a few of the manifestations of this crisis. Scientists argue that immediate action is required to mitigate the effects of climate change, emphasizing the need for a transition to renewable energy sources, the implementation of more stringent environmental policies, and the promotion of

sustainable living practices among the general population.”, we can identify Key-phrases like

“Climate Change”, “Rising average temperatures”, Declining biodiversity”, “Renewable energy sources” etc.

Leveraging the extracted key phrases, the summarization model then constructs a concise summary that encapsulates the main points of the text:

"Addressing climate change, a paramount global challenge, necessitates urgent actions such as adopting renewable energy, enforcing robust environmental policies, and encouraging sustainable living to counteract rising temperatures, extreme weather, and biodiversity loss."

In this example, key-phrase tagging efficiently filters through the text to highlight its most important points, enabling the creation of a summary that, despite the text being short, remains rich in information and accurately reflects the original text's themes.

- iii. Parts of Speech (POS) – Part of Speech tagging is a task in Natural Language Processing (NLP) that involves classifying words in a text with its correct part of speech such as nouns, verbs, adjectives, etc. This process is essential for NLP tasks like sentiment analysis, machine translation, etc. (Singh et al, 2013)

In the sentence "Running late, she decided to book a cab instead of taking the bus to the airport."

"Running (VERB)", "late (ADVERB)", "she (PRONOUNN)", "decided (VERB)", "to(PART)", "book(VERB)", "a(DETERMINER)", "cab(NOUN)", "in stead(ADVERB)", "of(PREPOSITION)", "taking(VERB)", "the(DETERMINER)", "bus(NOUN)", "to(PREPOSITION)", "the(DETERMINER)", "airport(NOUN)"

The example elaborates how POS annotation assigns a label to each word in the sentence based on the role in the sentence.

- iv. Sentiment analysis: Sentiment analysis involves categorizing pieces of text according to the sentiment expressed in them, typically as “positive”, “negative” or “neutral”. This is done to enable machines understand and interpret subjective information in human language (Asher, Benamara, & Mathieu, 2008).
- v. Intent annotation: Intent annotation plays an important role in the development of chatbots, virtual assistants, and various AI-based applications. It involves the process of identifying and categorizing the intention behind a user's message into predefined labels or categories. This review covers the foundational concepts, methodologies, recent

advances, and challenges associated with intent annotation (Weld, Huang, Long, Poon, & Han, 2021).

- vi. Document classification: Document classification is a process that aims to assign labels or tags to characters, sentences, queries, paragraphs, or documents. Document classification has a wide range of applications that includes sentiment analysis, content moderation, intent classification (Kalra, Kashyap, & Kaur, 2022) etc.

2.1.2 Annotation methods

Annotation in machine learning entails a spectrum of methodologies, each with its unique advantages and application areas. From annotating data manually to semi-automated tools and the use of crowdsourcing or bootstrapping approaches, the landscape of NLP annotation is both diverse and nuanced. This chapter goes in-depth on some annotation methodologies.

1. Manual Annotation: Manual annotation is a method which consists of reading a particular preselected document and providing additional information or assigning labels in the form of the annotations (Pernelle. 2016). Manual annotations can be done at any level, this includes, a sentence, a phrase, a word or even a character (Neves and Ševa 2021, 146). A new project created in annotation typically involves defining an annotation schema, formulating comprehensive guidelines, collecting adequate documents, preprocessing documents based on the task, training experts for annotation tasks and then finalizing on the corpus (Neves and Ševa 2021, 146). Choosing a tool for annotation is also not an easy task due to the large number of annotation tools along with its advantages and disadvantages. The decision to select a tool involves trying different tools to find out the best one that suits the use case. This process usually fails due to lack of documentation about the tool, installation of the tool owing to technical issues among others.
2. Semi-automatic Annotation: Semi-automatic annotation combines human insights with computational algorithms to annotate data (Pernelle. 2016). This process best represents the middle ground between manual annotation and automated annotation methods. The steps involved in semi-automatic annotation involves the process of collecting a dataset, then a computational algorithm is executed to pre-annotate the dataset. The algorithm may be a rule-based algorithm, a machine learning algorithm, or a large language model. The human annotator then reviews the pre-annotated data, correct errors or refine the annotation (Bonet-Jover et al. 2023).

3. Crowdsourcing generally involves distributing tasks across several individuals, often through an online tool (Finin et al. 2010). This approach can significantly accelerate the annotation process and is particularly useful for large-scale projects. On the other hand, bootstrapping methods, as applied in creating extensive annotated corpora like AraSenCorpus, leverage a small set of manually annotated data to train models that can then annotate larger datasets. AraSenCorpus demonstrates the effectiveness of semi-supervised learning using deep learning techniques to maintain high quality in sentiment analysis tasks (Al-Laith et al. 2021).

2.2 Large language models

Language is important for humans to communicate and express themselves, as well as engage with technologies. The necessity for generalised models arises from the increasing requirement for machines to manage linguistic tasks, such as translation, summarization, information retrieval, conversational exchanges, and so on (Naveed et al., 2024). Language models have recently experienced substantial advancements, mainly because of the use of transformers, improved processing power, and the abundance of extensive training data. These advancements have resulted in a ground-breaking change by allowing the invention of LLMs that can achieve a level of performance comparable to that of humans on different activities. A large language model (LLM) is a language model that can produce versatile language production and perform many natural language processing tasks, such as classification, sentiment analysis among others (Brown et al., 2020, Devlin et al., 2019.). LLMs develop these skills by obtaining statistical correlations from textual sources through a computationally demanding process of self-supervised and semi-supervised training. Language models (LLMs) can be utilized for text generation, which is a type of generative artificial intelligence (AI), by taking an input text and iteratively forecasting the subsequent token or word (Devlin et al., 2019.).

2.2.1 Transformer-based LLM

The transformer-based architecture is commonly regarded as the foundational component of large language models (LLMs). This architecture is intended for neural networks to properly handle and process sequential data. This architecture does not employ recursive procedures. Instead, it utilizes an attention mechanism to ascertain global input-output interdependencies. This has led to the development of new model sizes and improved performance levels, enabling significantly enhanced parallelization and decreased training durations in natural

language processing (NLP) (Wang, Li, & Smola, 2019). Most competitive neural sequence transduction models possess an encoder-decoder architecture. According to Vaswani et al. (2017) the encoder transforms an input series of symbol representations $(x_1, \dots, x_n)$ into a sequence of continuous representations $z = (z_1, \dots, z_n)$. Upon receiving z , the decoder then produces an output sequence $(y_1, \dots, y_m)$ of symbols sequentially, one element at a time. At each stage, the model operates in an auto-regressive manner, utilising the previously generated symbols as supplementary input for the subsequent generation. These are the components of an architecture-based LLM;

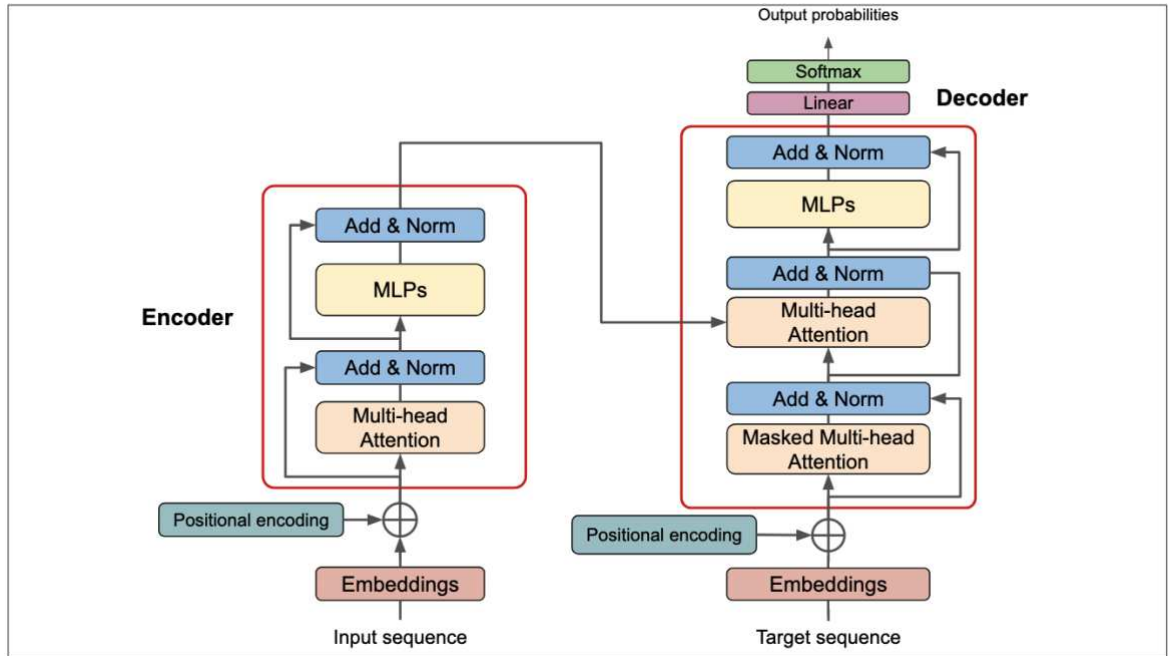


Figure 1: Transformer architecture. Adapted from BlueberryML

1. **Input embeddings:** The ML models utilise user-provided tokens as training data, but they are only capable of processing numerical information. Therefore, it is imperative to convert these textual inputs into a numerical representation called "input embeddings." The input embeddings are quantitative representations of words that machine learning models can process afterwards. These embeddings serve a similar purpose as a dictionary, aiding the model in comprehending the semantic meaning of words by organising them in a mathematical space where similar phrases are in proximity. The model is trained to construct these embeddings such that vectors of identical dimensions correspond to words with comparable semantic significance (Raiaan et al., 2024).

2. **Positional embeddings:** The word order in a sentence is crucial in the field of Natural Language Processing (NLP) for determining the meaning of a statement. Neural networks, in general, lack a fundamental understanding of the sequence of inputs. To address the problem, positional encoding can be used to represent the position of each word in the input sequence as a set of integers. The Transformer model incorporates integer inputs and input embeddings. Additionally, the model incorporates positional encoding within the transformer architecture, which enhances the ability to understand the word order in sentences. This results in the generation of grammatically accurate and semantically appropriate output (Raiaan et al., 2024).
3. **Encoder:** The encoder is an integral component of the neural network responsible for processing the input text and producing a sequence of concealed information. Next, it employs a sequence of self-attention layers to generate many concealed states that depict the input text at varying levels of abstraction. The encoder is utilized across multiple layers of the transformer (Raiaan et al., 2024).
4. **Outputs:** Through the training process, the decoder develops the capacity to forecast the subsequent word by examining the preceding words. In this scenario, the output sequence is displaced by one position towards the right. As a result, the decoder can utilize the preceding words (Raiaan et al., 2024).
5. **Output embeddings:** Input embeddings, consisting of textual information, that are not acknowledged by the model. Hence, it is necessary to convert the output into a format referred to as "output embedding." Like input embeddings, output embeddings also undergo positional encoding, which allows the model to comprehend the sequential arrangement of words inside a sentence. Within the field of machine learning, the loss function is employed to assess the disparity between a model's prediction and the desired objective value. Loss functions are crucial for intricate language models. The loss function adjusts a specific part of the model to enhance accuracy by minimizing the difference between predictions and targets. The modification enhances the overall efficiency of the model. The loss function is computed throughout the training process, and the model parameters are adjusted accordingly. During the inference phase, the output text is generated by associating the predicted probability of each token in the model with the corresponding term in the vocabulary (Raiaan et al., 2024).
6. **Decoder:** The decoder utilizes positional encoding to process input and output embeddings. Model decoders generate output sequences based on

encoded input sequences, while the decoder simultaneously learns to anticipate the subsequent word from the preceding words during the training phase (Raiaan et al., 2024).

7. **Linear Layer and Softmax:** Once the decoder has generated the output embedding, the linear layer maps to a higher-dimensional space. Converting the output embedding back into the original input space is a necessary step. The softmax function produces a probability distribution for each output word in the created vocabulary, enabling us to build output tokens with probabilities (Raiaan et al., 2024).

2.3 BERT architecture

BERT (Devlin et al. 2019) is a multi-layer bidirectional Transformer encoder that understands semantic and syntactic language information. The model has two primary configurations: BERT_{BASE}, which has 12 layers, 768 hidden units, and 12 self-attention heads, and BERT_{LARGE}, which consists of 24 layers, 1024 hidden units, and 16 self-attention heads. BERT's primary distinction is in its implementation of bidirectional self-attention, enabling it to evaluate context from both sides, in contrast to the unidirectional attention utilised in models such as GPT.

Input/Output Representations: BERT's input representation is designed to handle both single sentences and sentence pairs in a single token sequence. This is achieved by using special tokens like [CLS] and [SEP]. The first token of any input sequence is always [CLS], used for classification tasks. Sentence pairs are separated by the [SEP] token. BERT uses WordPiece embeddings with a vocabulary of 30,000 tokens. Each token in the input sequence is represented by a sum of its token embedding, segment embedding (indicating whether it belongs to sentence A or B), and position embedding (Devlin et al. 2019).

2.3.1 Training BERT

BERT is pre-trained using two unsupervised tasks:

1. **Masked Language Model (MLM):** In this task, some percentages of the input tokens are randomly masked, and the model is trained to predict the original tokens based on their context. This allows BERT to learn bidirectional representations by conditioning on both the left and right context.
2. **Next Sentence Prediction (NSP):** This task helps the model understand the relationship between two sentences. For each pre-training example, BERT

predicts whether the second sentence (B) follows the first sentence (A) or if it is just a random sentence from the corpus.

2.3.2 TinyBERT architecture

TinyBERT (Jiao et al., 2019) is a distilled variant of the BERT model designed to make it more efficient while preserving performance on natural language processing tasks. TinyBERT utilizes a multi-layer bidirectional Transformer encoder like BERT but with fewer layers and smaller hidden sizes, making it more compact and faster. This is accomplished by a Transformer distillation method that transfers knowledge from a large "teacher" BERT model to a smaller "student" TinyBERT model. The distillation is done in two stages:

1. General Distillation Stage: This stage distils the general knowledge from the pre-trained BERT model. The student TinyBERT learns to mimic the behaviour of the teacher BERT without fine-tuning on specific tasks. It captures the general-domain knowledge that BERT has learned during its pre-training phase on large-scale text corpora.
2. Task-specific Distillation Stage: After general distillation, TinyBERT undergoes further distillation tailored to specific tasks using a fine-tuned BERT model as the teacher. This involves a data augmentation strategy where new training examples are created, allowing TinyBERT to better capture the nuances of the specific tasks.

TinyBERT achieves around 96.8% of the performance of its teacher model BERT_{BASE} on the GLUE benchmark while being 7.5 times smaller and 9.4 times faster in inference.

2.4 GPT architecture

GPT-4 (OpenAI 2023) is a large language model developed by OpenAI, building on the architecture of its predecessors like GPT-3. It employs a multi-layer Transformer architecture, which is designed to predict the next word in a sequence based on the context of the preceding words. The model is pre-trained using a massive amount of publicly available and licensed data, including text from the internet, books, and articles.

GPT-4 is built using the Transformer model architecture (Vaswani et al. 2017), which consists of multiple layers of attention mechanisms. Each layer processes input sequences to produce output sequences that capture relationships between

words. Unlike earlier models, GPT-4 features enhanced scalability, allowing it to process complex sequences of text more efficiently and accurately.

Input/Output Representations: The input to GPT-4 can be a single sentence or multiple sentences packed together. It uses a token-based representation where each word or sub-word unit is encoded into tokens. These tokens are then transformed into embeddings that represent the semantic meaning of the input text. The model utilizes these embeddings to generate an output sequence, which can be a continuation of the input text, an answer to a question, or even code generation.

The model's training and architecture make it versatile for various applications, including question answering, summarization, and complex reasoning tasks.

2.5 Prompt engineering

Prompt engineering is an essential component of utilising large language models (LLMs) like GPT-4 for text annotation jobs. A prompt is a collection of directives given to a large language model (LLM) that customises and/or improves its functionalities. A prompt can affect subsequent interactions with, and the output produced by a large language model (LLM) by establishing explicit rules and guidelines for the conversation based on initial parameters. A prompt establishes the framework for the dialogue, indicating to the LLM the pertinent information and the expected structure and substance of the output (White et al., 2023). For instance, a prompt may dictate that a large language model exclusively produce code adhering to a specific coding style or programming paradigm. Similarly, it could stipulate that an LLM must identify specific keywords or phrases in a created document and furnish supplementary information pertinent to those terms. The introduction of these standards enables prompts to produce more structured and nuanced outputs, hence assisting a wide range of software engineering tasks within the framework of LLMs.

2.6 Review of text annotation applications

Based on research, some annotation tools were selected and rated based on the criteria selected. The annotation tools were: WordFreak, MMAX2, BRAT, Doccano, Prodigy, Textinator and Label Studio. Each of these tools offers unique features tailored to specific annotation needs:

1. According to Morton and LaCivita (2003), WordFreak is a versatile and extensible platform for linguistic annotation, designed to support a broad

range of languages and annotation tasks. Developed in Java, WordFreak enables both manual and automated annotation processes, making it adaptable for different linguistic research needs. One of its most notable features is a plug-in architecture that allows users to customize the tool's functionality without requiring recompilation, facilitating the addition of new visualizations and annotation specifications as well as the integration of automatic annotators. WordFreak supports various annotation types, including constituent parse structures, dependency annotations, named-entity recognition, and coreference annotation. This flexibility makes it applicable across different linguistic domains and tasks. The tool is designed with multilingual support, initially covering languages such as English, Chinese, and Arabic, which allows researchers to perform cross-linguistic analyses within a single platform. Additionally, WordFreak is open source, released under the Mozilla Public License 1.1, and available for customization and extension, which further enhances its usability in diverse research settings. Overall, WordFreak's feature set, including its extensibility, support for multiple annotation types, and compatibility with various languages, makes it a valuable tool for both linguistic researchers and NLP practitioners.

2. The MMAX2 (Müller & Strube, 2006) annotation tool is a Java-based application designed for the multi-level annotation of linguistic data, particularly at the discourse level. It employs a flexible stand-off XML data format, allowing annotations to be stored separately from the primary text, which facilitates complex annotation tasks without altering the original data. MMAX2 is highly customizable, enabling users to define their own annotation schemes and visualization styles to suit specific project requirements. Its user-friendly interface supports the annotation of various linguistic phenomena, including coreference resolution, discourse structure, and other relational data. Additionally, MMAX2 integrates with external tools and scripts, enhancing its functionality and adaptability for diverse linguistic research needs.
3. The BRAT Rapid Annotation Tool (BRAT) (Stenetorp et al., 2012) is a web-based application designed for efficient text annotation, particularly in natural language processing (NLP) tasks. Developed to facilitate the creation of structured annotations, BRAT supports a wide range of annotation types, including entities, relations, and events. Its interface allows annotators to visualize and edit annotations directly within the text, enhancing the annotation process's intuitiveness and speed. BRAT's flexibility and configurability make it suitable for various annotation projects, from simple entity recognition to complex event extraction.

Additionally, it offers features such as support for multiple languages, making it a valuable tool for both research and industry applications in NLP.

4. Doccano (Nakayama, Kubo, Kamura, Taniguchi, & Liang, 2018) is an open-source text annotation tool designed to facilitate the creation of labelled data for various natural language processing (NLP) tasks. It supports multiple annotation types, including text classification, sequence labelling (e.g., named entity recognition), and sequence-to-sequence tasks (e.g., machine translation and text summarization). Doccano's web-based interface allows users to create projects, upload datasets, define labels, and annotate text efficiently. Key features include collaborative annotation capabilities, multi-language support, and a RESTful API for integration with machine learning pipelines.
5. Prodigy (Explosion AI, n.d.) is a downloadable annotation tool designed for tasks in natural language processing (NLP), machine learning, and computer vision. Developed by Explosion AI, Prodigy offers a modern data development experience, enabling users to efficiently define, train, and evaluate custom AI systems. It supports various annotation tasks, including named entity recognition, text classification, object detection, and image segmentation. Prodigy is highly extensible, allowing users to build custom workflows and integrate with existing machine learning pipelines. It operates entirely under the user's control, ensuring full data privacy.
6. Label Studio (HumanSignal, n.d.) is an open-source data labelling platform developed by HumanSignal. It supports a wide range of data types, including text, images, audio, video, and time series, making it versatile for various machine learning and data science projects. Users can create customized labelling interfaces using a straightforward configuration setup, allowing for tailored annotation workflows. Label Studio also offers integration capabilities with machine learning models, enabling functionalities such as pre-labelling and active learning to enhance annotation efficiency. Its flexibility and extensibility make it a valuable tool for both researchers and practitioners in the field of data annotation.
7. Textinator (Kalpakchi & Boye, 2022) is an open-source, internationalized annotation and evaluation tool designed for Natural Language Processing (NLP) tasks. Developed by Dmytro Kalpakchi and Johan Boye, it offers a customizable web interface that supports a wide range of annotation tasks, including span labelling, relation annotation, text classification, and more. One of Textinator's distinguishing features is its support for multiple languages in the user interface, which lowers the entry threshold for domain experts and helps control for unintended biases introduced by

using only English-speaking annotators. The tool also facilitates human evaluations by allowing the import of surveys with potentially hundreds of evaluation items in one click. Textinator's flexibility and extensibility make it a valuable resource for researchers and practitioners in the NLP field.

Over the past years, there has been a significant increase in the number of annotation tools, offering a wide range of options to pick from. These criteria not only cover the requirements of this thesis but are also informed by insights from existing literature (Kalpakchi & Boye, 2022) and official documentation where available. Surface-level examination of the source code was also conducted, but no direct communication occurred with the tools' authors or creators. Each criterion is further divided into specific questions designed to capture essential functional aspects that are critical to a successful annotation workflow. Tools were evaluated against these criteria, and a scoring system was used to quantify their performance in each area. Each question was assigned one point, resulting in a maximum possible score of 4 for User Interface, 4 for Collaboration, 3 for High Quality Annotation and 4 for Semi-Automation:

1. User Interface: A dynamic and user-friendly UI is essential for users to effectively visualize and annotate text. For users with varying levels of technical expertise, an intuitive UI can facilitate accessibility and ease of use. According to Kalpakchi & Boye (2022), task type coverage support for different annotation tasks such as named entity recognition, text classification, and relation annotation is crucial for adaptable UIs in annotation tools. Evaluating if each tool supports a broad range of tasks provides insight into its flexibility across projects. Here are some critical questions that will guide the assessment:
 - i. How intuitive is the application for first-time users?
 - ii. Can users easily navigate through the application without extensive training?
 - iii. How adaptable is the UI to different annotation projects or user preferences?
 - iv. In collaborative settings, how does the UI distinguish between different user roles or permissions?
2. Collaboration: Referring to Kalpakchi and Boye's (2022) study, tools that provide real-time monitoring of annotation progress, such as completion tracking and inter-annotator agreement scores, were found to support teams in maintaining quality and consistency in collaborative settings. Efficient collaboration is key to the success of text annotation projects, especially when dealing with large datasets or requiring annotations from experts in different domains. Collaborative features should enable

seamless communication, real-time updates, and conflict resolution among annotators. To evaluate these aspects, consider the following questions:

- i. Does the application support real-time collaboration and how are changes reflected for all users?
 - ii. Are there commenting and discussion tools available for annotators to communicate effectively within the application?
 - iii. Does the application offer role-based access control and customizable permissions for different users?
 - iv. How does the system manage conflicting annotations and what mechanisms are in place for resolution?
3. High-Quality Annotation: Incorporating internationalization capabilities (Kalpakchi & Boye, 2022) helps ensure that annotations are consistently high-quality across languages and cultural contexts. Tools supporting multilingual annotations allow for accurate data labelling by diverse annotator groups, helping to reduce linguistic and cultural biases.:
- i. How efficient and user-friendly are these features?
 - ii. Are there tools or features that assist in minimizing human error?
 - iii. How does the application promote consistent annotations among different users, especially in cases of subjective interpretation?
4. Semi-automation: Integrating machine learning assistance in annotation can expedite the process, especially for large datasets. Evaluating semi-automation features involves assessing the relevance, accuracy, and efficiency of automated suggestions. Tools with customizable semi-automation features, such as configurable pre-annotation settings (Kalpakchi & Boye, 2022) allow users to tailor automation to their needs, balancing efficiency with human oversight. To evaluate the effectiveness and impact of semi-automation within an annotation application, these questions need to be considered:
- i. How accurate and relevant are the automated suggestions provided by the application?
 - ii. Do they significantly reduce the time and effort required for manual annotation?
 - iii. How does the application ensure that human expertise remains paramount?
 - iv. How much time is saved on average with the use of semi-automation compared to fully manual annotation processes?

Table 1: Comparison of text annotation applications based on key criteria

Annotation application	User Interface	Collaboration	High Quality Annotation	Semi-automation	Total
WordFreak	1	0	1	0	2
MMA2	2	1	1	0	4
BRAT	3	2	2	1	8
Doccano	4	3	3	2	12
Prodigy	4	4	4	3	15
Textinator	2	1	1	1	5
Label Studio	4	3	3	3	13

Source: Compiled by the author

Each tool's final score is the sum of its scores across the four criteria, with higher scores indicating more robust features across diverse annotation needs. Prodigy and Label Studio achieved the highest scores, reflecting their advanced capabilities in collaboration, high-quality annotation support, and semi-automation. These tools stood out as the most suitable options for complex, collaborative annotation projects. In contrast, tools like WordFreak and MMA2 scored lower, indicating they are best suited for simpler, single-user annotation tasks without extensive automation or collaboration needs.

This is a breakdown of one of the applications (WordFreak):

1. User Interface (1/4): Basic UI, functional but not very adaptable and requires some learning curve.
2. Collaboration (0/4): Limited collaboration tools and only supports primarily single user.
3. High-Quality Annotation (1/3): Minimal tools for error reduction, no multilingual support.
4. Semi-Automation (0/4): No semi-automation or machine learning assistance.

2.7 Evaluation metrics

Evaluating the performance of a Large Language Model (LLM) in text annotation tasks necessitates the comparison of its generated annotations with a gold standard. The "gold standard" for a natural language processing task is a dataset of natural language texts annotated by people to provide accurate solutions for that specific task (Kovář, Jakubíček, & Horák, 2016). Presented herein is a comprehensive methodology for juxtaposing LLM-generated annotations against a gold standard.

Additional metrics were utilised for the internal assessment of the fine-tuning process to obtain deeper insights into the models' learning and generalisation across classes. The precision, recall, and F1 score for each class were computed to guarantee that the fine-tuning process was thorough and focused on improving performance across all classes.

1. Precision, defined as the ratio of accurately predicted positive observations to the total anticipated positives, offers insight into the model's capacity to correctly detect positive cases (Powers, 2011).

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

2. Accuracy measures the percentage of documents (or entities, in the case of NER) correctly classified by the model (Powers, 2011).

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of documents}}$$

3. Recall is the ratio of accurately predicted positive observations to the total observations in the actual class, emphasising the model's capacity to recognise all potential positive cases (Powers, 2011).

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

4. The F1-score, as the weighted average of precision and recall, equilibrates these two metrics, especially in scenarios where one may hold greater importance than the other (Powers, 2011).

$$\text{Precision} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{True Positives} + \text{False Positives}}$$

The application of these indicators enabled a comprehensive assessment, guaranteeing that the models were not prejudiced towards the majority class and that the fine-tuning procedure significantly improved the models' performance across all classes.

3 Methodology

This chapter aims to explain the methodological framework and theoretical underpinnings that inform the design and execution of a text annotation application. To design this tool, it is crucial to implement an approach that conforms to established best practices in human-computer interaction, collaborative systems, and text processing semi-automation (Nielsen, 1993; Fort et al., 2014). This chapter outlines the processes for creating an application that is intuitive, facilitates collaboration among annotators, and employs semi-automation to alleviate human labour, thereby satisfying contemporary requirements in the domain of text annotation.

The study's methodology comprises the following steps. Each segment of effort has been predicated on the preceding outcome. The technique emphasises identifying core requirements:

1. Requirements gathering
2. User interface (UI) design
3. System Design
4. Choice of Technologies
5. Selection of large language models
6. Implementation
7. Evaluation

3.1 Requirements gathering

The initial phase of requirements gathering involved a comprehensive review of existing text annotation tools, including Brat (Stenetorp et al., 2012), Doccano (Nakayama, Kubo, Kamura, Taniguchi, & Liang, 2018) and Prodigy (Explosion AI, n.d). These tools offer essential functionalities, such as text selection, labelling, and collaborative annotation capabilities foundational to any text annotation platform. Analysing these tools provided insight into baseline functional requirements and revealed areas for enhancement. Each tool embodies key components of text annotation but also presents usability limitations that can inform the development of a more effective and user-centred application.

For example, Brat is widely recognized for its web-based interface, yet lacks flexible support for complex, customized workflows (Stenetorp et al., 2012). Doccano (Nakayama, Kubo, Kamura, Taniguchi, & Liang, 2018) supports multi-user collaboration and is accessible via a web interface, making it popular for team-based projects. However, it is limited in terms of automation features. Prodigy (Explosion AI, n.d), facilitates rapid annotation by suggesting labels based

on machine learning models, though it is largely suited to users already proficient in programming and lacks collaboration features for team-based projects. By analysing these tools, we identified key areas for improvement, such as user-friendliness, collaboration features, and integration with machine learning models for semi-automated annotation suggestions.

3.2 User interface design

The design of an effective User Interface (UI) is crucial for ensuring that the text annotation application is intuitive, adaptable, and supports a range of annotation tasks. A well-designed UI enhances usability, reduces cognitive load, and promotes efficient task completion, all of which are especially important for annotation tools that need to accommodate users with varying levels of expertise. This section outlines the thought process that will go into the design choices and usability:

1. **Layout and accessibility:** The UI layout will follow usability heuristics outlined by Nielsen (1993), such as consistency, visibility, and feedback. Consistency will be achieved by placing common actions (e.g., save, undo) in familiar locations, like a top or side toolbar, and ensuring that icons and buttons use clear, universally recognizable symbols. This layout consistency will reduce the mental effort required to locate functions, improving overall efficiency.
2. **Colour and contrast:** Colour choices are made to enhance readability and focus, avoiding bright or distracting colours that can lead to visual fatigue. According to Shneiderman & Plaisant (2004), optimal colour contrast and a minimalist palette can help users focus on the primary tasks, reducing distractions. Therefore, a neutral background with accent colours for labels and highlights will be used to ensure that annotations are visually distinct without overwhelming the user.
3. **Menu and tool placement:** Tools frequently used by annotators such as the labelling palette, entity selector, and validation options are placed in a fixed menu to ensure they are always accessible. This arrangement aligns with principles of direct manipulation and efficiency, allowing users to perform annotation tasks without excessive navigation (Shneiderman & Plaisant, 2004). The placement of these tools in proximity to the main text area also minimizes eye and mouse movement, improving speed and comfort.

3.3 System design

System design is a crucial phase in application development that defines the architecture, components, interactions, and data flow within a system. A well-planned system design is essential to ensure scalability, maintainability, and performance, especially in applications that require real-time processing, user interactions, and efficient data management. According to Bass, Clements, & Kazman (2003), system design should balance functional requirements (such as data storage, processing, and user interaction) with quality attributes like performance, reliability, and usability. In the context of a text annotation application, the system must support high user interactivity, real-time data processing, collaboration, and integration with machine learning components for semi-automation.

3.4 Choice of technologies

For a text annotation application that supports efficient, collaborative, and semi-automated workflows, the chosen technology stack plays a central role in fulfilling functional and non-functional requirements. Each component, frontend, backend, database, file processing, and machine learning integration was selected for its strengths in performance, scalability, and usability. The following section explains each technology choice in depth, providing specific reasons why it is well-suited for this application.

3.4.1 Choice of technologies

Nuxt 3, a powerful framework based on Vue.js, was selected for the frontend to provide a structured, server-rendered (SSR) frontend with improved performance and SEO capabilities, which is especially beneficial in web-based applications where response times and user experience are critical (Ribeiro, 2022). Nuxt 3 supports a modular component-based architecture that enables reusable UI components, ideal for an annotation tool where repetitive UI elements (e.g., annotation panels, toolbars) can be defined once and reused throughout the interface.

Nuxt's SSR capability ensures faster initial page loads and better search engine indexing, improving the perceived performance of the application, especially for users with slower internet connections. Additionally, Nuxt 3's integration with Vuex (for state management) and Vue Router makes it easier to manage application state and route users across different annotation tasks, facilitating a

smoother user experience in navigating and managing text annotation workflows (Sharma, 2018).

3.4.2 Backend

Django was chosen as the backend framework for its robustness, security, and "batteries-included" philosophy, making it a comprehensive solution for building scalable and secure applications. Django's ORM (Object-Relational Mapping) simplifies database interactions, allowing seamless integration with PostgreSQL for storing structured annotation data. The Django REST Framework (DRF) provides tools for developing a RESTful API, enabling efficient communication between the Nuxt frontend and the backend, which is essential for managing the asynchronous, interactive nature of annotation tasks (Holovaty & Kaplan-Moss, 2009).

Django's built-in features, such as user authentication and authorization, provide secure access control for collaborative annotation, where different user roles (e.g., annotators, reviewers, and administrators) may require distinct permissions. This robust security framework is particularly valuable in applications that handle sensitive user data or confidential documents (Forcier, Bissex, & Chun, 2009).

3.4.3 Database

PostgreSQL, an open-source relational database known for its reliability and advanced querying capabilities, was selected to store the application's structured data, including user profiles, annotations, and document metadata. PostgreSQL's support for ACID (Atomicity, Consistency, Isolation, Durability) compliance ensures data integrity, which is crucial for collaborative applications where multiple annotators may interact with the same dataset (Momjian, 2001).

PostgreSQL also supports complex data types like JSON, allowing for storing semi-structured data, such as ML model responses or metadata that might vary across documents. This flexibility is useful for an annotation tool where data can range from straightforward text labels to complex annotation metadata. Additionally, PostgreSQL's robust indexing and search capabilities ensure efficient data retrieval, which is essential for handling large volumes of annotated data and supporting fast query performance during text retrieval and analysis.

3.5 Model selection

In selecting large language models (LLMs) for Natural Language Processing (NLP) tasks, two models were considered for named entity recognition and sequence labelling tasks: TinyBERT model and GPT-4. Both models offer distinct advantages depending on the task and objectives of the project, and they were chosen to complement each other for different NLP tasks.

1. BERT model (Devlin et al. 2019) is engineered to pretrain deep bidirectional representations from unlabelled text by simultaneously conditioning on both left and right contexts across all layers. The pre-trained BERT model can be fine-tuned with a single additional output layer to develop state-of-the-art models for many tasks, including question answering and language inference, without significant task-specific architectural alterations. The BERT model was selected due to its strong performance on a wide range of NLP tasks, particularly text classification and entity recognition. The BERT model excels in:
 - i. Text Classification and Token-Level Tasks: BERT, being a transformer-based model pre-trained on large text corpora, is particularly well-suited for token-level tasks such as Named Entity Recognition (NER), and sentence-level tasks like text classification. Its ability to understand the context of individual words in sentences makes it ideal for semantic analysis.
 - ii. Fine-tuning BERT for various downstream tasks is uncomplicated due to the self-attention mechanism of the Transformer, which can accommodate tasks requiring both individual texts and text pairs. In tasks involving text pairings, such as paraphrasing or entailment, BERT utilizes self-attention to manage bidirectional cross attention between the texts by encoding the joined text pair. For diverse NLP tasks, specific inputs and outputs are integrated into BERT, and the model is fine-tuned in an end-to-end manner. In token-level tasks, such as sequence labelling or question answering, token representations are transmitted to an output layer. The [CLS] token representation is utilised for classification tasks, such as sentiment analysis. Fine-tuning BERT is computationally economical relative to pre-training and may be accomplished in approximately one hour with a Cloud TPU or several hours on a GPU.
 - iii. Bidirectional Context Understanding: In contrast to earlier models such as GPT, which analyse text unidirectionally (either left-to-right or right-to-left), BERT operates bidirectionally, considering both the left and right context of each word. BERT excels at comprehending the contextual meaning of words, which is essential for tasks such as

Named Entity Recognition (NER), where adjacent words assist in identifying named entities (e.g., differentiating "Apple" as a corporation from some fruit).

Given these strengths, BERT was the logical choice for initial model development where high accuracy on structured text and domain-specific NLP tasks was paramount.

2. GPT-4 is a Transformer-based model pre-trained to anticipate the subsequent token in a document, utilising both publicly accessible data and data licensed from external providers. The model was further refined by Reinforcement Learning from Human Feedback. Its benefits included:
 - i. Zero-Shot and Few-Shot Learning: A primary rationale for choosing GPT-4 is its exceptional efficacy in zero-shot and few-shot learning tasks. This indicates its ability to generalise to tasks for which it has not received explicit training, rendering it a formidable instrument for text generation and intricate question answering without the necessity of a substantial annotated dataset.
 - ii. Contextual Comprehension and Reasoning: GPT-4 is recognised for its capacity to produce cohesive and contextually precise writing, even when addressing intricate and nuanced subjects. This rendered it optimal for tasks such summary generation, responding to open-ended enquiries, and discerning purpose from text.
 - iii. In comparison to open-source models such as BERT, GPT-4 has consistently excelled in general language comprehension, generation, and reasoning across several benchmarks, especially in tasks like stance detection and relevance categorisation.

3.6 Implementation

The implementation of the text annotation application involved integrating the selected tools, frameworks, and large language models into a cohesive system. This process included frontend development, backend integration, and machine learning (ML) model deployment to achieve semi-automation for annotation tasks.

3.6.1 Frontend development

Using Nuxt 3, the frontend components were developed to provide a seamless user experience. Core UI elements, such as text labelling panels, entity selectors, and collaborative features, were implemented as reusable Vue.js components. This

modular approach facilitated scalability, ensuring that the UI could easily accommodate additional features like custom annotation workflows.

3.6.2 Backend integration

The Django backend was implemented to handle user authentication, data management, and collaborative features. The Django REST Framework (DRF) provided a RESTful API that connected the frontend with backend services, enabling real-time updates for collaborative annotations. For example, user roles (annotator, reviewer, administrator) were managed using Django's robust permissions system, ensuring secure and role-specific access.

3.6.3 Large language model integration

TinyBERT was fine-tuned for Named Entity Recognition (NER) tasks, while GPT-4 was utilized for zero-shot learning scenarios such as generating annotation suggestions for unfamiliar datasets. These models were exposed via APIs to the backend, which served as an intermediary between the ML services and the frontend.

3.6.4 Database implementation

PostgreSQL was configured to handle structured and semi-structured data. Table schemas were designed to store annotation metadata, user interactions, and model predictions efficiently. Features like JSONB fields in PostgreSQL were used to store and query semi-structured data, such as GPT-4-generated annotation suggestions.

3.7 Evaluation

The evaluation of the text annotation application aims to evaluate its performance across four key dimensions: a user-friendly interface, collaborative features, the assurance of high-quality annotations, and the minimisation of human annotation efforts through semi-automation. The evaluation process includes the author of the thesis and two student annotators to replicate varied usage scenarios and guarantee a thorough assessment. This dual participation guarantees that the assessment incorporates both expert and end-user viewpoints.

The usability of the interface is evaluated by both the author of the thesis and student annotators performing a range of annotation tasks, including labelling entities in texts of varying lengths and complexities. The assessment follows

Nielsen's usability heuristics (Nielsen, 1993), focusing on ease of navigation, consistency, and responsiveness. These tasks are designed to test whether annotators can intuitively interact with key features, such as labelling tools and annotation suggestions. The layout is tested for its ability to minimize cognitive load, as consistency in tool placement is a critical usability factor (Shneiderman & Plaisant, 2004). Additionally, the colour schemes and contrast levels are examined for their effectiveness in maintaining readability and highlighting annotations without inducing visual fatigue. Adjustments, such as optimizing icon sizes and enhancing help tooltips, are planned to address potential usability issues identified during testing.

The application's collaboration capabilities are tested through simulated multi-user interactions, involving both the author of the thesis and student annotators. Multiple annotator accounts are created to mimic concurrent annotation workflows. Role-based permissions are analysed to confirm proper access control, ensuring that specific users can only access or modify data relevant to their role.

Annotation quality is a central focus of the evaluation and is assessed by comparing the application's outputs against established datasets, such as CoNLL-2003 (Sang & De Meulder, 2003). Precision, recall, and F1-score are used as evaluation metrics to measure the accuracy of annotations, while inter-annotator agreement is evaluated through simulated reliability tests to ensure consistency across repeated annotations (Artstein & Poesio, 2008). High precision and recall scores are anticipated, reflecting the system's ability to facilitate detailed and accurate labelling. Furthermore, the tool's ability to minimize annotation inconsistencies is evaluated by analysing errors in model-generated and manually adjusted annotations.

The automation features integrated into the application are evaluated for their impact on reducing manual annotation workload. Tasks are performed both with and without automation to measure time savings and workload reduction. The accuracy of machine-suggested annotations, powered by TinyBERT (Jiao et al., 2020) and GPT-4 (OpenAI, 2023), is assessed against ground truth datasets. TinyBERT is expected to excel in structured tasks such as named entity recognition, offering precise token-level suggestions, while GPT-4 is anticipated to provide high-quality zero-shot and few-shot annotation suggestions for complex or unfamiliar datasets. Automation features are expected to significantly reduce manual effort, as prior research indicates that leveraging pre-trained models can streamline repetitive tasks and improve efficiency (Devlin et al., 2019; Brown et al., 2020).

Overall, this evaluation is designed to determine the application's success in meeting its objectives. The findings guide further refinements to the tool and establish its viability as a solution for contemporary text annotation challenges. By employing rigorous testing methods and leveraging established benchmarks, the evaluation ensures that the application adheres to high standards of usability, collaboration, and automation.

4 Results

This section defines the results of the text annotation application created to facilitate sentiment analysis and named entity recognition. The development of this application necessitated a structured approach that commenced with the definition of requirements, guaranteeing that the system fulfilled the needs based on research in the processing and annotation of textual material. The results chapter will detail the design decisions, encompassing the overarching architecture and the user interface, illustrating how each component was developed to enable smooth interaction and effective annotation processes. Essential application elements, such as annotation capabilities, model integration, and data management systems, are comprehensively defined to demonstrate the entire implementation procedure. Every component was developed and executed with a focus on usability, precision, high quality annotations, collaboration, establishing a robust framework for efficient and intelligent text analysis utilising large language models. This section defines the results of the text annotation application created to facilitate sentiment analysis and named entity recognition tasks.

4.1 Requirements analysis

The development of the text annotation application is grounded in an analysis of technical requirements. These requirements were informed by the findings from the review of existing tools and literature in Chapter 2. By addressing the limitations and gaps in current tools, this application is designed to provide an efficient, user-friendly, and scalable solution for text annotation tasks. The requirements define the functionalities the system must deliver, including the services it offers, and the constraints under which it operates. Functional requirements are particularly critical, as they specify the services the system must perform, its responses to user inputs, and its behaviour in specific scenarios. In some cases, functional requirements also delineate what the system must not do, ensuring a robust and predictable user experience (Sommerville, 2013).

4.1.1 Functional requirements

This section will discuss the criteria to guarantee that the text annotation application delivers an efficient, user-friendly, and scalable solution for annotating text data:

1. User management: The requirement for user management stems from the need for role-based access control and collaboration, which is only partially supported by tools like Prodigy. The system must support different types of users (e.g., annotators, reviewers, administrators) and manage their roles and permissions.
 - i. Users can create accounts, log in, and log out securely.
 - ii. The system must allow role-based access control where administrators can assign roles such as annotators and reviewers.
2. File upload and data management: Current tools such as Brat and Doccano are limited in their ability to handle diverse file formats or provide detailed feedback during file uploads. To address this, the system includes:
 - i. Users can upload text files in various formats (e.g., .csv, .txt).
 - ii. The system should handle file metadata (e.g., file name, upload date, file size).
 - iii. The system must validate file formats before uploading and notify users of any issues (e.g., unsupported file formats).
3. Text annotation: Manual annotation in tools like Brat is tedious and time-consuming, while Prodigy offers automation but requires significant technical expertise. This system integrates manual and automatic annotation workflows to provide flexibility and efficiency.
 - i. Users can manually annotate text by highlighting text and applying labels (e.g., Named Entities, sentiment, categories).
 - ii. The system must integrate pre-trained and fine-tuned LLMs (such as BERT and GPT-4) to perform automatic annotations.
 - iii. Users can view model-suggested annotations and modify them as needed.
4. Model fine-tuning: Unlike existing tools, which often lack user-friendly interfaces for customizing ML models, this system allows users to fine-tune LLMs on domain-specific datasets.
 - i. Users can upload custom datasets to fine-tune LLMs for specific annotation tasks.
 - ii. The system provides a user interface for specifying fine-tuning parameters (e.g., number of epochs, learning rate).
 - iii. Users can save and store fine-tuned models for future annotation tasks.
5. Collaboration and review workflow: Collaborative annotation is a key limitation in tools like WordFreak, which are designed for individual use. This system explicitly supports team-based workflows.
 - i. Users can create annotation projects and invite others to collaborate.
 - ii. Users can add comments or feedback on documents for review and discussion.

6. Exporting annotations: The system must allow users to export the annotated data in various formats for external use.
 - i. Users can export annotated datasets in multiple formats, including json, csv, and jsonl.
7. Error handling and validation: The system must gracefully handle errors and provide meaningful feedback to users.
 - i. Users should receive feedback when annotation models fail or produce errors, with suggestions for resolution.
 - ii. The system validates uploaded files for correct formats and encoding types before processing.

4.1.2 Non-functional requirements

Non-functional requirements are limitations on the services or functions provided by the system. They encompass temporal limitations, restrictions on the development process, and constraints dictated by standards. Non-functional requirements typically pertain to the system in its entirety, rather than to specific features or services. (Sommerville, 2011). This section highlights the non-functional requirements specific to building a text annotation application:

1. Security: The system must ensure that all data, including text files, annotations, and user information, is securely stored, and transmitted.
 - i. The system must implement role-based access control (RBAC) to limit access to certain features based on user roles.
 - ii. The system should integrate with authentication mechanisms such as OAuth 2.0 or JWT for secure user authentication.
2. Usability: The system must be intuitive and easy to use for both technical and non-technical users.
 - i. The system should have a simple and clean user interface, with clear navigation and minimal learning curve.
 - ii. Users should be able to perform annotation tasks with minimal clicks (e.g., one-click access to most-used features).
3. Maintainability: The system must be easy to maintain and update without causing significant downtime or issues for users.
 - i. The system codebase should follow a modular design, allowing for the easy addition of new features or models without affecting existing functionality.
4. Extensibility: The system must allow for future extension and integration with new models or annotation tools.

- i. The system must be designed to allow for the integration of additional LLMs (e.g., BERT, GPT-4) without major architectural changes.

4.1.3 Use case diagram

A use case diagram is a UML diagram that visually depicts the interactions between users (actors) and systems to accomplish certain objectives or functions (use cases). It offers a comprehensive summary of the interactions between various users (including administrators, annotators, and external systems) and the software system, as well as the system's functional capabilities.

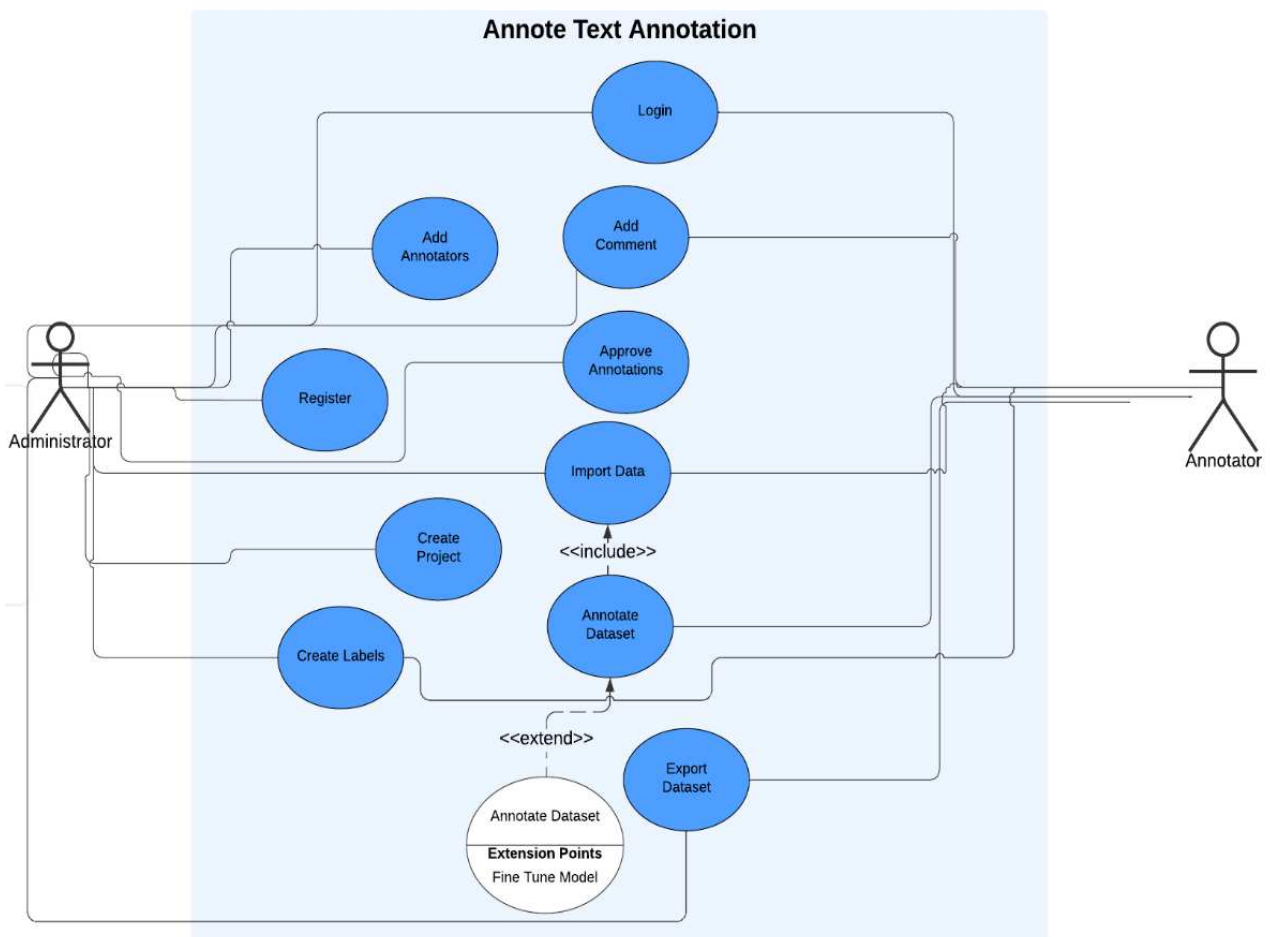


Figure 2: Use case diagram for text annotation application

The diagram above represents the functional requirements of the Text Annotation Application. The system is designed to support text annotation tasks by interacting with different users (actors) like Administrator and Annotator.

1. Register: This use case involves Administrators registering new users into the system, such as new annotators who will work on the dataset.
2. Login: Both Administrators and Annotators need to log into the system. This use case is common across all actors to authenticate and start using the system.
3. Create Project: Administrators can create new projects, which would include setting up the data and annotation tasks.
4. Add Annotators: After creating a project, Administrators can add annotators to the project to begin working on specific annotation tasks.
5. Create Labels: Administrators define the annotation labels or categories (e.g., Named Entities, Sentiment) that annotators will use to tag text data. This is a key preparation step for the annotation process.
6. Import Data: Both Administrators and Annotators are involved in importing datasets into the system. The imported data will be annotated by the annotators later.
7. Annotate Dataset: This is the primary use case for the Annotators. They perform the actual annotation of text using pre-defined labels or using the assistance of large language models (LLMs) such as BERT or GPT-4. This step involves labelling entities or adding sentiment to text documents.
 - i. <<include>> Relationship: The Annotate Dataset use case includes the Import Data use case. This means that before annotators can annotate text, the data must be imported into the system first. Every time annotation is done, data import must have been completed, making this relationship essential.
8. Add Comment: Annotators can also add comments during the annotation process to provide feedback or additional context on specific annotations. This enhances collaboration among team members.
9. Approve Annotations: Administrators or reviewers are responsible for approving the annotations submitted by the annotators. This use case ensures the quality and consistency of the annotated data before it is finalized.
10. Export Dataset: Once annotations are completed and approved, the Administrator can export the dataset for further analysis or machine learning model training. This use case is primarily linked to the

Administrator but may involve others for exporting in specific formats such as JSON, CSV, or JSONL.

- i. <<extend>> Relationship: The Export Dataset use case is an extension of the Annotate Dataset use case. This indicates that exporting the dataset is optional and occurs only after the annotations have been completed. The extension allows flexibility in how the system handles the final output.

4.2 User interface design

The design of the text annotation application emphasizes simplicity, accessibility, and ease of navigation, making it user-friendly and approachable for both new and experienced users. The following section evaluates the main aspects of the UI design, including the choice of colours, the layout structure, and the organization of menus.

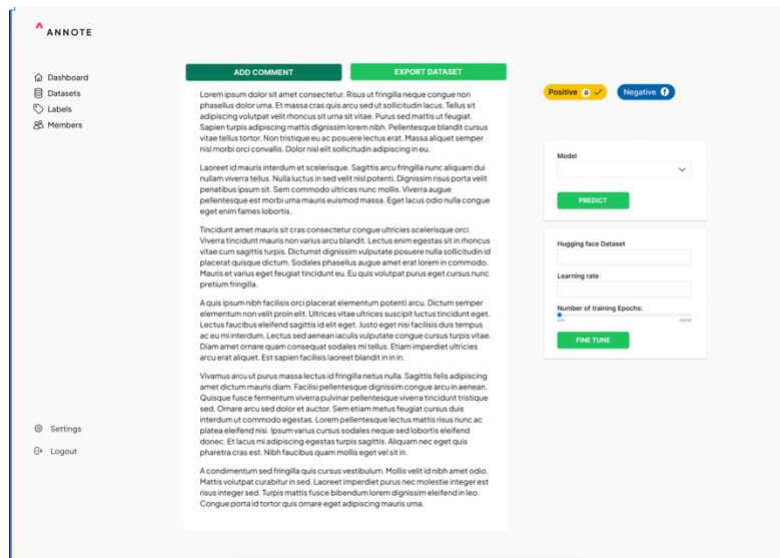


Figure 3: Figma design for sentiment analysis task

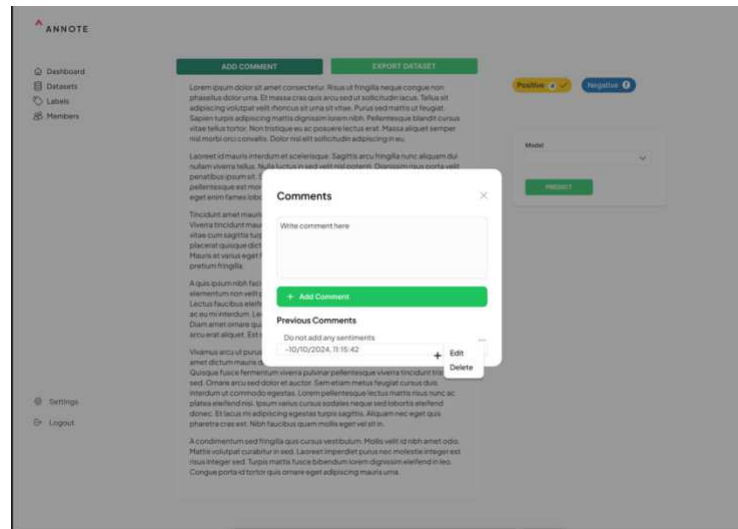


Figure 4: Figma design for commenting on an annotation task

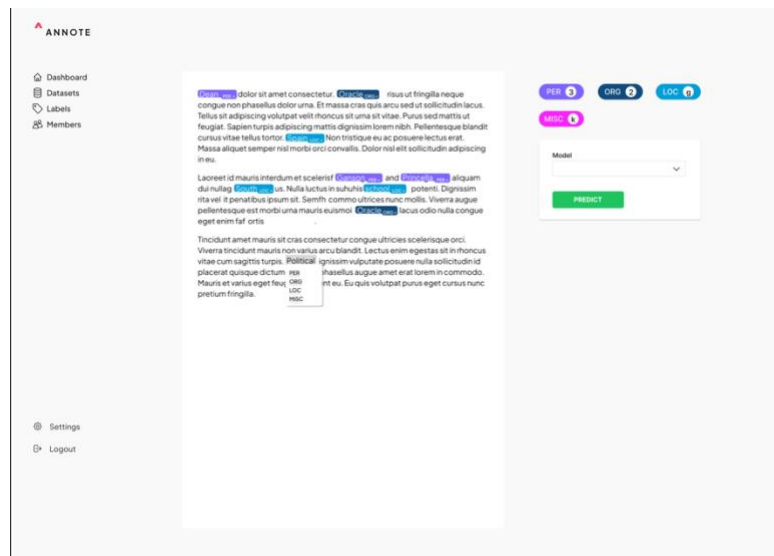


Figure 5: Figma design for NER task

1. Figma design for sentiment analysis: The dashboard offers a clean and intuitive layout. A navigation menu on the left provides quick access to sections like "Datasets," "Labels," and "Members," while the main workspace is divided into three panels namely;
 - i. Text panel: Displays the text to be annotated, allowing users to highlight and label entities or sentiments.
 - ii. Action buttons: The "Add Comment" and "Export Dataset" buttons are placed at the top for quick access to collaboration and export features.

- iii. Automation and fine-tuning panel: The right-hand panel allows users to select a pre-trained model (e.g., GPT-4 or TinyBERT), generate predictions, and fine-tune models by specifying parameters like learning rate and number of epochs.
2. Figma design for commenting on a task: The comments feature enhances collaboration among annotators by enabling them to provide feedback or clarify ambiguities directly within the application.
 - A "Comments" button opens a modal window where users can write and view comments.
 - Each comment is timestamped, and options to edit or delete comments are available, ensuring effective communication and traceability during collaborative workflows.
 - This feature plays a vital role in ensuring high-quality annotations by facilitating discussions and feedback among team members.
3. Figma design for NER task: The Named Entity Recognition (NER) interface is optimized for tagging specific entities within a document, such as "Person," "Location," "Organization," and "Miscellaneous."
 - Entities are highlighted in distinct colours (e.g., blue for "Person" and green for "Location") to enhance visual clarity.
 - Users can manually adjust or accept predictions provided by the integrated models, making it easier to achieve accurate annotations.
 - The interface dynamically updates counts of labeled entities, providing users with real-time feedback on their progress.
 - This design effectively balances automation with manual oversight, ensuring both efficiency and accuracy in the annotation workflow.

Although limited in scope, these designs provide crucial guidance during the early stages of development, particularly in defining the application's aesthetic direction and usability principles. By focusing on the essential pages, the designs ensure that the application's foundational elements colour schemes, layout structure, and error handling were effectively implemented.

4.3 System design

System design emphasises the architecture and foundational structure of the system. Establishing the design framework from a macro system perspective lays the foundation for a more intricate design of each component of the system. The system architecture delineates the layering and partitioning necessary to contain functions within subsystems. The system architecture also dictates the physical platform for the system and the distribution of various services. This section

discusses the various diagrams to be created and gives a visualization on the application to be built.

1. Activity Diagram: An Activity Diagram illustrates the progression of activities or tasks inside a process. It demonstrates the sequential execution of several actions, encompassing decision points, loops, and concurrent processes.

The use case for performing named entity recognition powered by large language models (LLMs) like BERT or GPT-4 will be captured using an activity diagram.

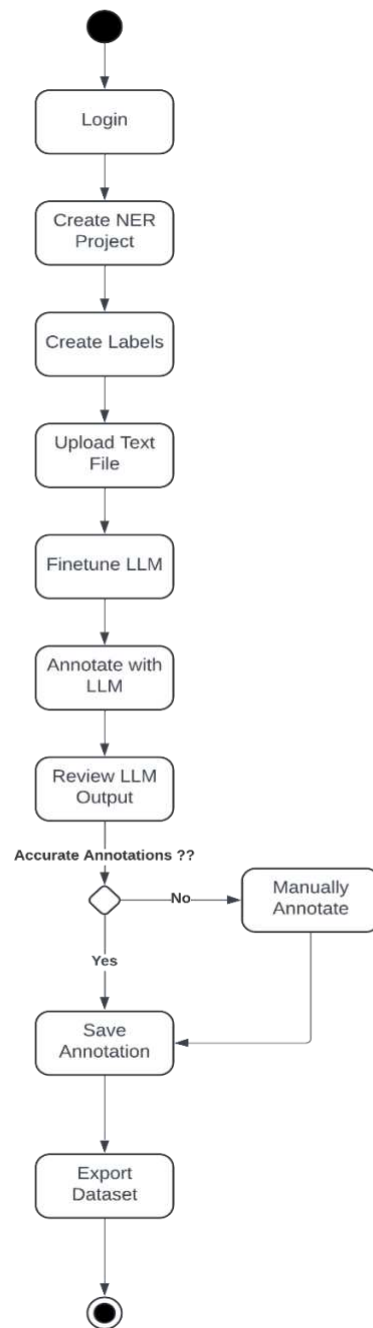


Figure 6: Activity diagram for annotating text

- i. Start: The activity begins when the user accesses the application and logs in.
- ii. Create NER project: The user creates a Named Entity Recognition project.
- iii. Create Labels: The user creates labels which are going to be used for the annotation.
- iv. Upload Text File: The user uploads a file for annotation. The system validates and stores the file.
- v. Finetune LLM: The user enhances the model 's performance for specific datasets or domains, they can initiate the fine-tuning process
- vi. Annotate with LLM: The system sends the text data to the pre-trained large language model (such as BERT or GPT-4). The model processes the text and returns a list of predicted named entities with their respective labels (e.g., Person, Organization, Location).
- vii. Review LLM output: The user reviews the NER predictions made by the model. The predictions are highlighted and displayed in the annotation interface.
- viii. Decision point – Accurate Annotations ??:
 - a. If the user is satisfied with the model's suggestions, they proceed to save the final annotations.
 - b. If the user is not satisfied, they can manually adjust the annotations (e.g., adding missing entities, correcting mislabelling, etc.).
- ix. Export Annotated Data (Optional): If the user wishes to export the annotated dataset, they can choose the export format (CSV, JSON, JSONL) and download the annotated text.

4.4 Implementation

This chapter provides a comprehensive overview of the implementation process for the text annotation application, including the integration of fine-tuned models and the processing of files. It details how the large language models were integrated into the system, and how the application utilizes these models for text annotation. Code snippets and visual representations are included to demonstrate key components of the application. The implementation process emphasizes the proposed methodologies, aligning them with the theoretical framework discussed in this chapter.

4.4.1 Database connection

The backend was developed using the python programming language utilizing the Django Rest Framework. To enable Django to interact with PostgreSQL, the

psycopg2 package is required and the database configurations were defined in the “settings.py” file

```
backend > backend > settings.py > ...
159
160
161
162
163 # Database
164 # https://docs.djangoproject.com/en/4.0/ref/settings/#databases
165
166 DATABASES = {
167     'default': {
168         'ENGINE': 'django.db.backends.postgresql',
169         'NAME': os.environ.get('DB_NAME'),
170         'USER': os.environ.get('DB_USER'),
171         'PASSWORD': os.environ.get('DB_PASSWORD'),
172         'HOST': os.environ.get('DB_HOST', 'localhost'),
173         'PORT': os.environ.get('DB_PORT', '5432'),
174     }
175 }
176
```

Figure 7: Code for defining connection to database

4.4.2 Registration

To register a new user, one only clicks on "Create free Account" on the homepage of the application. The registration form will allow the user to provide their first name, last name, username, email, and password. Upon the user's click on the register button, the input will undergo validation prior to the invocation of the registerUser() method.

The RegisterUser class is defined as a subclass of APIView from Django REST Framework. It handles HTTP POST requests for user registration.



Figure 8: Backend code for processing registration request

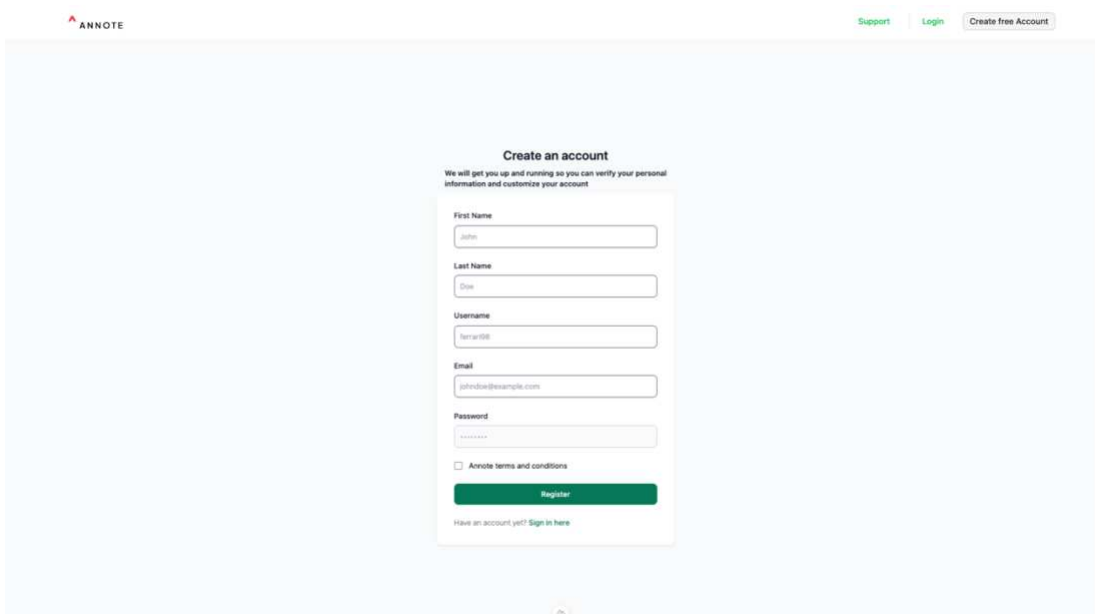


Figure 9: UI for registering user

4.4.3 Login

The Pinia store (auth.js) manages the authentication state, handling login, logout, and token management. The login function Sends a POST request to the /auth/login endpoint with the user credentials. If the response is successful, the access and refresh tokens are stored, and the isLoggedIn state is set to true.

The login function in the frontend script handles the process of authenticating a user by sending their credentials to the backend and then navigating them to the home page upon successful login.

4.4.4 Project creation

The project creation module is an integral part of the text annotation application, allowing users to define and configure new projects according to their requirements. Key fields include project name, description, project type, and several project-specific settings such as tag management, classification options, and sharing settings.

The form data is sent to the backend API via a POST request. The Authorization header includes the user's authentication token, ensuring secure communication. The API endpoint (<http://127.0.0.1:8000/project/projects>) processes the request and creates a new project record in the database.

Figure 10: UI for creating a project

This class-based view in Django provides a robust interface for managing projects with the following key features:

1. **Listing and Searching Projects:** Users can view their associated projects, search by name or description, and sort based on various fields.
2. **Creating Projects:** Authenticated users can create new projects. The view automatically assigns the creating user as the project admin.
3. **Project admins can delete multiple projects in one request.** Due to polymorphic constraints, individual deletion is performed to avoid errors.

4.4.5 Label creation

This section details the implementation of a feature for creating label types in the application. This feature allows users to define and customize labels, which are essential for annotating text data in various projects such as sentiment analysis or sequence labelling.

The form provides an interface for users to define and customize label types, including fields such as label name, suffix key, and colour settings. The form's layout is structured to be user-friendly and responsive, ensuring ease of use across different devices.

The form is powered by a series of methods that handle user inputs, form submission, and data interaction with the backend.



Figure 11: UI for creating a label

4.4.6 Import dataset

The dataset import interface for the text annotation application is designed to streamline the process of adding new datasets to the platform. The interface, developed using Vue.js, provides users with a user-friendly environment to select file formats, upload files, and configure various dataset options. This section details the interface's components and the logic that governs its operation.

The interface consists of a structured form where users can interact with various input fields and options:

1. **File Format Selection:** Users can select the format of the dataset they wish to upload from a dropdown menu. This selection is essential as it dictates the structure and encoding of the uploaded data. Supported formats

include various common data types such as CSV, JSON, and others tailored for specific annotation tasks.

2. **Encoding Selection:** Once a file format is selected, users can choose the appropriate text encoding for the dataset, such as UTF-8. This ensures that the data is correctly interpreted during the upload process, preventing issues related to character encoding.
3. **File Upload:** The interface integrates the file-pond component, which provides a drag-and-drop zone for users to upload their files. This component supports chunk uploads and displays progress indicators, offering feedback on the status of the file upload. Users can upload multiple files simultaneously and remove files if necessary.
4. **Dataset Configuration Options:** Additional configuration options, such as column data, labels, and delimiters, are available based on the selected file format. These options allow users to customize how the dataset is processed and utilized within the application.
5. **Submit Button:** Once the necessary fields and configurations are set, users can submit the dataset using the "Save" button. The button is dynamically enabled or disabled based on the form's validity, ensuring that all required fields are correctly filled before submission.

The dataset import interface is a crucial component of the text annotation application, enabling efficient data onboarding. By providing an intuitive and responsive interface, it reduces the complexity of importing datasets, allowing users to focus on the annotation process. The integration of robust feedback mechanisms, dynamic validations, and automated task tracking ensures that the import process is smooth and error-free, contributing to an overall seamless user experience.

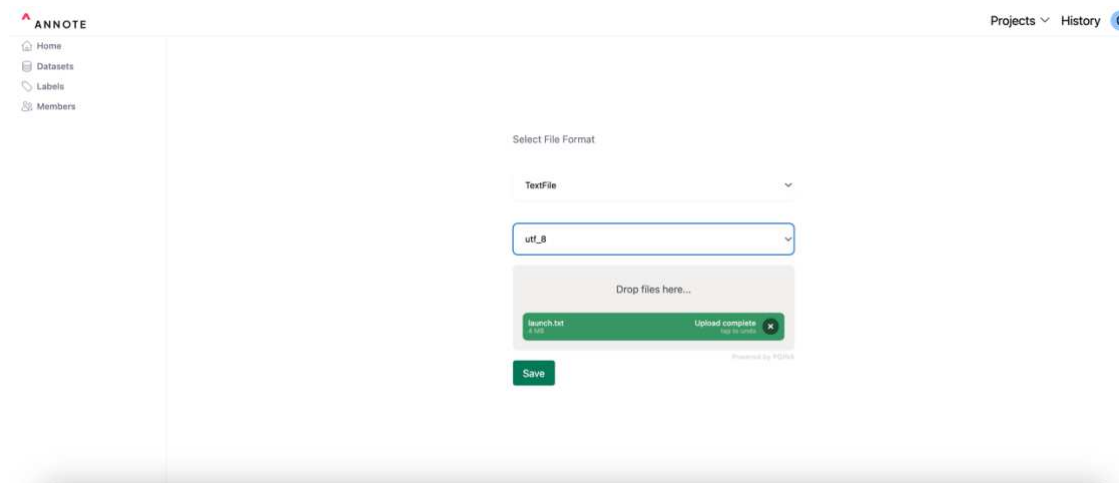


Figure 12: UI for uploading text file

This method is a critical part of the backend architecture for the text annotation application. It provides the following benefits:

1. **Asynchronous Processing:** By using Celery tasks, the import process can be handled asynchronously, allowing the application to process large datasets without blocking the main application workflow.
2. **Scalability and Error Handling:** Automatic retries, backoff strategies, and detailed error logging ensure that the dataset import process is robust and can recover from transient issues.
3. **Modular and Reusable Design:** The use of abstract classes and helper functions makes the code modular and easy to extend. New dataset types or file formats can be added with minimal changes to the existing code.
4. **User Feedback and Logging:** Errors are returned in a structured format, allowing the frontend to provide meaningful feedback to the user. This improves the user experience by making the dataset import process transparent and predictable.

4.4.7 Adding annotators

The "Add User" feature within the text annotation application enables project administrators to add new members to a project with designated roles. This interface, located under the "Members" section, allows administrators to input essential user details such as first name, last name, username, and email address. Additionally, administrators can assign specific roles to users, choosing from predefined types like "project_admin," "annotator," and "annotation_approver."

This role-based access ensures that each user's permissions align with their responsibilities within the project, enhancing the application's collaborative capabilities. By allowing fine-grained role assignments, this feature supports efficient workflow management, as different roles cater to specific tasks, such as annotation creation or approval.

Figure 13: Frontend for adding an annotator

4.4.8 Annotator comments

The "Comments" feature in the text annotation application allows users to add contextual notes and feedback on individual text samples. This functionality is especially useful for collaborative projects, where annotators and reviewers can provide insights, clarifications, or additional instructions regarding specific annotations. Users can write and submit new comments, which are then displayed in a list below the input field, organized chronologically for easy reference. Each comment entry includes a timestamp, providing a clear record of when feedback was added.

In addition to adding new comments, users have the option to edit or delete existing ones, ensuring flexibility in managing feedback throughout the annotation process. These actions are accessible via a dropdown menu next to each comment, allowing quick updates without cluttering the interface. This feature supports a streamlined communication process among team members, promoting a collaborative approach to refining annotations and improving the overall quality of the data.

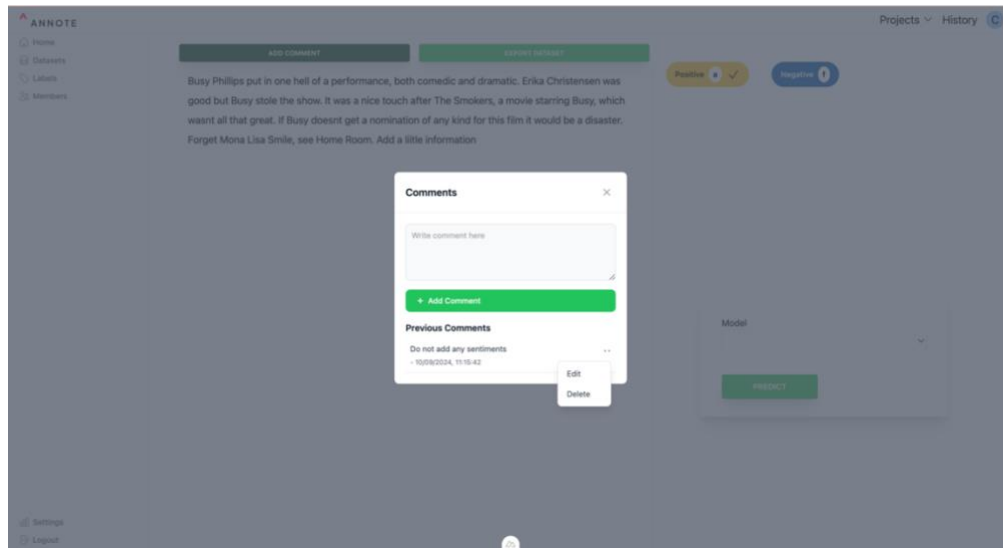


Figure 14: Frontend for comments functionality

4.4.9 Finetuning BERT for named entity recognition

The dataset employed for fine-tuning BERT to perform a name entity recognition task was the CoNLL-2003 dataset. The CoNLL-2003 data files consist of four columns separated by a single space. Each word has been positioned on an own line, with a blank line succeeding each sentence. Every line comprises a word, a part-of-speech (POS) tag, a syntactic chunk tag, and a named entity tag. The English data was sourced from the Reuters Corpus. This corpus comprises Reuters news articles. from August 1996 to August 1997. Data spanning ten days was extracted from the files corresponding to the end of August 1996 for the training and development set. The texts in the test set were sourced from December 1996. The pre-processed raw data encompasses September 1996.

The interface offers a "Fine-Tune" option specifically for customizing models based on Hugging Face datasets. Users can choose a Hugging Face dataset (e.g., "conll2003") and adjust parameters such as the learning rate and the number of training epochs using an interactive slider. This fine-tuning capability is limited to Hugging Face datasets, ensuring compatibility and ease of integration within the annotation environment. With a single click, users can initiate the fine-tuning process, optimizing model performance to better suit the characteristics of their data.

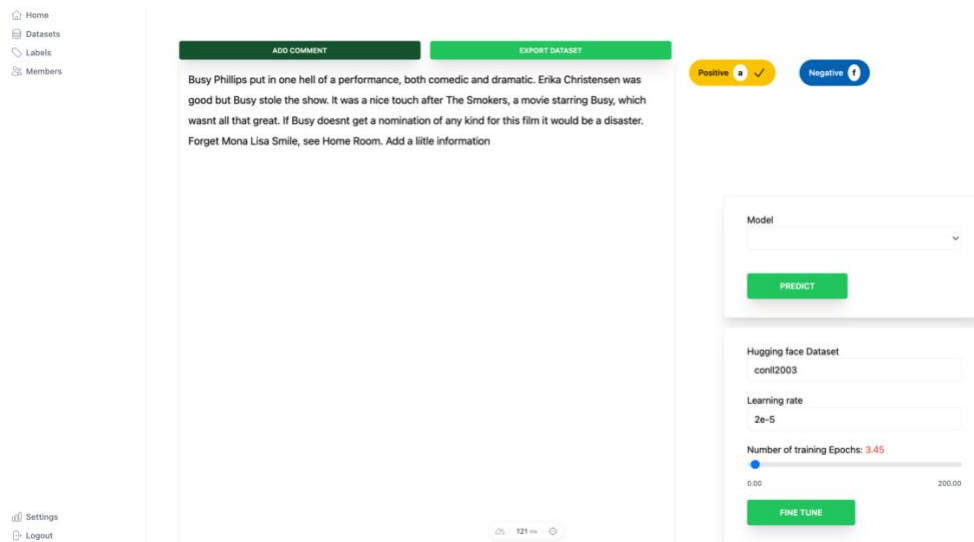


Figure 15: Frontend for fine-tuning

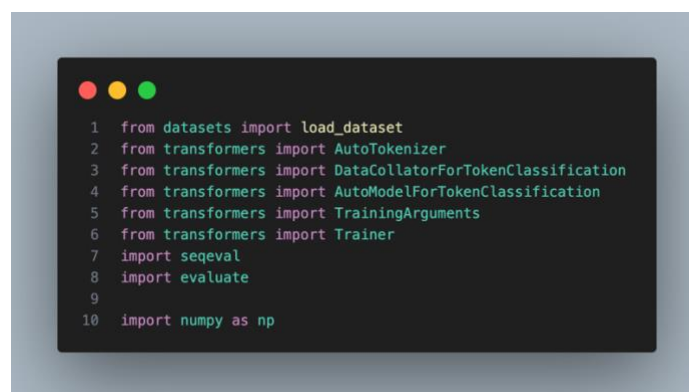


Figure 16: Backend code for importing modules

The provided code snippet above outlines the process of building, fine-tuning, and evaluating a Named Entity Recognition (NER) model using the Hugging Face transformers library and the datasets library. The workflow involves multiple steps, from loading and tokenizing the dataset to training the model and evaluating its performance.

A screenshot of a code editor with a dark background and light-colored text. The code is written in Python and consists of six lines. Line 1 is a comment: '# Load the dataset'. Line 2 is 'raw_datasets = load_dataset(dataset)'. Line 3 is an empty line. Line 4 is an empty line. Line 5 is 'ner_feature = raw_datasets["train"].features["ner_tags"]'. Line 6 is an empty line. The code is highlighted with a light blue background.

```
1 # Load the dataset
2 raw_datasets = load_dataset(dataset)
3
4
5 ner_feature = raw_datasets["train"].features["ner_tags"]
6
```

Figure 17: Backend code for loading dataset

A screenshot of a code editor showing the structure of the CoNLL 2003 dataset. The code is a Python dictionary representing a DatasetDict. It contains three keys: 'train', 'validation', and 'test'. Each key points to a Dataset object. The 'train' dataset has 14041 rows and features 'id', 'tokens', 'pos_tags', 'chunk_tags', and 'ner_tags'. The 'validation' dataset has 3250 rows and the same features. The 'test' dataset has 3453 rows and the same features. The code is highlighted with a light blue background.

```
DatasetDict({
  train: Dataset({
    features: ['id', 'tokens', 'pos_tags',
'chunk_tags', 'ner_tags'],
    num_rows: 14041
  })
  validation: Dataset({
    features: ['id', 'tokens', 'pos_tags',
'chunk_tags', 'ner_tags'],
    num_rows: 3250
  })
  test: Dataset({
    features: ['id', 'tokens', 'pos_tags',
'chunk_tags', 'ner_tags'],
    num_rows: 3453
  })
})
```

Figure 18: features for CoNLL 2003 dataset

The image above is an overarching container that organizes different subsets of the dataset. The training dataset has the following features:

1. **id**: This feature serves as a unique identifier for each data entry, which is useful for tracking and referencing specific samples.
2. **tokens**: These are the basic units of text, split by words or subwords, used as input to the model.
3. **pos_tags**: Part-of-speech tags provide syntactic information about each token, such as whether it is a noun, verb, adjective, etc.
4. **chunk_tags**: These tags group tokens into meaningful chunks, like noun phrases or verb phrases, providing additional syntactic structure information.
5. **ner_tags**: These tags label each token as belonging to a specific entity type (e.g., B-PER for the beginning of a person's name, I-PER for the continuation, O for outside any named entity).

The training dataset consists of 14,041 rows, each representing a sequence of tokens with associated labels, the validation dataset includes 3,250 rows. This subset is used to tune hyperparameters and validate the model's performance during training and the test dataset has 3,453 rows. This subset is also used to evaluate the final performance of the model after training is complete.

```

1  def align_labels_with_tokens(labels, word_ids):
2      new_labels = []
3      current_word = None
4      for word_id in word_ids:
5          if word_id != current_word:
6              # Start of a new word!
7              current_word = word_id
8              label = -100 if word_id is None else labels[word_id]
9              new_labels.append(label)
10         elif word_id is None:
11             # Special token
12             new_labels.append(-100)
13         else:
14             # Same word as previous token
15             label = labels[word_id]
16             # If the label is B-XXX we change it to I-XXX
17             if label % 2 == 1:
18                 label += 1
19             new_labels.append(label)
20
21     return new_labels
22
23
24 labels = raw_datasets["train"][0]["ner_tags"]
25 word_ids = inputs.word_ids()
26
27
28 def tokenize_and_align_labels(examples):
29     tokenized_inputs = tokenizer(
30         examples["tokens"], truncation=True, is_split_into_words=True
31     )
32     all_labels = examples["ner_tags"]
33     new_labels = []
34     for i, labels in enumerate(all_labels):
35         word_ids = tokenized_inputs.word_ids(i)
36         new_labels.append(align_labels_with_tokens(labels, word_ids))
37
38     tokenized_inputs["labels"] = new_labels
39     print(tokenized_inputs)
40     return tokenized_inputs
41
42
43 tokenized_datasets = raw_datasets.map(
44     tokenize_and_align_labels,
45     batched=True,
46     remove_columns=raw_datasets["train"].column_names,
47 )
48
49
50 data_collator = DataCollatorForTokenClassification(tokenizer=tokenizer)
51
52
53 batch = data_collator([tokenized_datasets["train"][i] for i in range(2)])
54
55
56 for i in range(2):
57     print(tokenized_datasets["train"][i]["labels"])

```

Figure 19: Backend code for finetuning model

The code above performs the task of preparing and aligning NER (Named Entity Recognition) labels with the corresponding tokens for training a model. A step-by-step explanation of the code is below:

1. “align_labels_with_tokens (labels, word_ids)” function aligns the original NER labels with the tokenized words by iterating over the word_ids generated by the tokenizer. For each word_id, it checks if it's different from the current word. If so, it marks the start of a new word and assigns the corresponding label from the labels list. For special tokens like [CLS] and [SEP] that do not correspond to any word, it assigns a label of -100. This is used to ignore these tokens during model training. If the label is in the format B-XXX (beginning of an entity), it converts it to I-XXX (inside of an entity) if the word is split into multiple tokens, ensuring consistent labeling across sub-tokens.
2. “tokenize_and_align_labels(examples)” function works by tokenizing input text and aligning labels with the tokens. The tokenizer processes each sentence in examples[“tokens”], using truncation=True to handle long sequences and is_split_into_words=True to maintain word boundaries. The align_labels_with_tokens function is called to align the original labels (examples[“ner_tags”]) with the tokenized words. The tokenized inputs are returned with the new aligned labels stored under the key “labels”
3. The tokenized_datasets variable stores the processed and tokenized version of the original raw_datasets with aligned labels. The map() function applies tokenize_and_align_labels to each example in the dataset, and removes unused columns like id and tokens.
4. The data_collator is used to collate a batch of tokenized data, ensuring that all sequences in the batch have the same length.

```

1 metric = evaluate.load("seqeval")
2 labels = raw_datasets["train"][0]["ner_tags"]
3 labels = [label_names[i] for i in labels]
4
5 predictions = labels.copy()
6 predictions[2] = "O"
7 print(metric.compute(predictions=predictions, references=labels))
8
9 def compute_metrics(eval_preds):
10     logits, labels = eval_preds
11     predictions = np.argmax(logits, axis=-1)
12
13     # Remove ignored index (special tokens) and convert to labels
14     true_labels = [[label_names[l] for l in label if l != -100] for label in labels]
15     true_predictions = [
16         [label_names[p] for (p, l) in zip(prediction, label) if l != -100]
17         for prediction, label in zip(predictions, labels)
18     ]
19     all_metrics = metric.compute(predictions=true_predictions, references=true_labels)
20     return {
21         "precision": all_metrics["overall_precision"],
22         "recall": all_metrics["overall_recall"],
23         "f1": all_metrics["overall_f1"],
24         "accuracy": all_metrics["overall_accuracy"],
25     }
26
27 id2label = {i: label for i, label in enumerate(label_names)}
28 label2id = {v: k for k, v in id2label.items()}

```

Figure 20: Backend code for calculating metrics

```

1 model = AutoModelForTokenClassification.from_pretrained(
2     model_checkpoint,
3     id2label=id2label,
4     label2id=label2id,
5 )
6
7 args = TrainingArguments(
8     "bert-finetuned-ner",
9     evaluation_strategy="epoch",
10    save_strategy="epoch",
11    learning_rate=learning_rate,
12    num_train_epochs=training_epochs,
13    weight_decay=0.01,
14    push_to_hub=True,
15    per_device_train_batch_size=4,
16    per_device_eval_batch_size=4,
17 )
18
19 trainer = Trainer(
20     model=model,
21     args=args,
22     train_dataset=tokenized_datasets["train"],
23     eval_dataset=tokenized_datasets["validation"],
24     data_collator=data_collator,
25     compute_metrics=compute_metrics,
26     tokenizer=tokenizer,
27 )
28 trainer.train()
29
30
31 trainer.push_to_hub(commit_message="Training complete", token=os.getenv("huggingface_token"))

```

Figure 21: Code for finetuning and publishing model

Here is a comprehensive explanation of the code above:

1. The seqeval metric is loaded to evaluate NER models. It is specifically used for computing metrics like precision, recall, and F1 score for sequence labeling tasks, making it suitable for evaluating NER performance.
2. The labels variable holds a sample set of labels from the training data, converting numeric labels to their corresponding names using label_names.
3. A copy of the labels is made and modified to create a sample predictions list.
4. The metric.compute function is used to compare the predictions with the references (true labels) and calculate evaluation metrics like precision, recall, and F1 score. This helps in verifying the metric calculation process.
5. A custom metric function for evaluation called “compute_metrics” was created to calculate evaluation metrics during model training and evaluation.
 - i. The logits are converted into label indices using np.argmax.
 - ii. Labels and predictions are filtered to remove special tokens (indexed as -100).
 - iii. The metric is computed using the seqeval metric, comparing predictions with true labels.
 - iv. The function returns a dictionary containing precision, recall, F1 score, and accuracy.
6. id2label and label2id dictionaries are created to map between numeric label indices and label names.
7. A pre-trained BERT model (bert-base-cased) is loaded and configured for token classification tasks. The model is set up with custom id2label and label2id mappings, enabling it to handle the specific labels used in the dataset.
8. The TrainingArguments method specifies configurations for the training process, such as the directory to save the model (bert-finetuned-ner), learning rate, number of training epochs, and batch sizes. evaluation_strategy="epoch" and save_strategy="epoch" indicate that the model should be evaluated and saved at the end of each epoch. The push_to_hub=True enables pushing the trained model to the Hugging Face Hub.
9. A Trainer object is created to handle the training and evaluation process. The training and validation datasets, data collator, metric computation function, and tokenizer are passed to the Trainer. The train() method starts the training process using the specified arguments and datasets.

10. After training, the model is pushed to the Hugging Face Hub, making it accessible for future use and inference.

After training of the model, the following results were observed:

Table 2: Model Training and Evaluation Metrics Across Epochs for BERT

Training Loss	Epoch	Step	Validation Loss	Precision	Recall	F1	Accuracy
0.106	0.5	1756	0.0832	0.8868	0.9244	0.9052	0.9788

Source: Compiled by author

4.4.10 Finetuning TinyBERT for sentiment analysis

The dataset employed for fine-tuning TinyBERT for the purpose of sentiment analysis was the IMDb dataset. The IMDb dataset is a binary sentiment analysis collection of 50,000 reviews from the Internet Movie Database (IMDb), categorised as positive or negative. The data fields are;

1. Text: a string feature
2. Label: a classification label, with possible values including neg , pos .

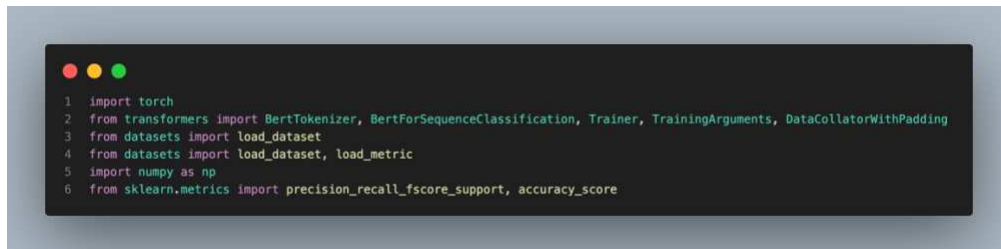


Figure 22: Backend code for importing modules for finetuning

The provided code snippet above outlines Hugging Face's transformers and datasets libraries are employed in combination with PyTorch to fine-tune a BERT model for sentiment analysis tasks. The process involves utilizing pre-trained models such as BertForSequenceClassification and tokenizing text data using BertTokenizer. The Trainer and TrainingArguments classes are used to manage the training configurations, including parameters such as learning rate, batch size, and the number of epochs, which simplifies the fine-tuning process.

```
1 def compute_metrics(eval_pred):
2     logits, labels = eval_pred
3     predictions = np.argmax(logits, axis=-1)
4     precision, recall, f1, _ = precision_recall_fscore_support(labels, predictions, average='weighted')
5     acc = accuracy_score(labels, predictions)
6     return {
7         'accuracy': acc,
8         'precision': precision,
9         'recall': recall,
10        'f1': f1,
11    }
12
13
14 def fine_tune_bert():
15     id2label = {0: "Negative", 1: "Positive", 2: "Neutral"}
16     label2id = {"Negative": 0, "Positive": 1, "Neutral": 2}
17     dataset = load_dataset("stanfordnlp/imdb")
18
19     tokenizer = BertTokenizer.from_pretrained("huawei-noah/TinyBERT_General_4L_312D")
20     model = BertForSequenceClassification.from_pretrained(
21         "huawei-noah/TinyBERT_General_4L_312D",
22         num_labels=3,
23         id2label=id2label,
24         label2id=label2id
25     )
26
27     def tokenize_function(examples):
28         return tokenizer(examples["text"], padding="max_length", truncation=True, max_length=128)
```

Figure 23: Backend code for finetuning model

```

1  tokenized_datasets = dataset.map(tokenize_function, batched=True)
2
3  data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
4
5  training_args = TrainingArguments(
6      output_dir="./results",
7      evaluation_strategy="epoch",
8      save_strategy="epoch",
9      learning_rate=2e-5,
10     per_device_train_batch_size=8,
11     per_device_eval_batch_size=8,
12     num_train_epochs=3,
13     weight_decay=0.01,
14     load_best_model_at_end=True,
15     metric_for_best_model="accuracy",
16     save_total_limit=1,
17 )
18
19 trainer = Trainer(
20     model=model,
21     args=training_args,
22     train_dataset=tokenized_datasets["train"],
23     eval_dataset=tokenized_datasets["test"],
24     data_collator=data_collator,
25     compute_metrics=compute_metrics,
26     callbacks=[CustomCallback()],
27 )
28 trainer.train()
29
30 metrics = trainer.evaluate()
31 print(metrics)
32 model.save_pretrained("./my_model")
33 tokenizer.save_pretrained("./my_model")

```

Figure 24: Code for finetuning BERT

Here is an in-depth explanation of the code above:

1. The `compute_metrics` function is designed to evaluate the performance of a BERT model for sequence classification tasks. It takes evaluation predictions (`eval_pred`) as input, which consist of logits (raw model outputs) and labels (ground truth). The logits are converted to class predictions using `np.argmax` to select the class with the highest probability. The function then computes key evaluation metrics precision, recall, F1 score, and accuracy using scikit-learn's `precision_recall_fscore_support` and `accuracy_score`. These metrics are returned as a dictionary to track the model's performance across different

evaluation metrics, ensuring a comprehensive evaluation of the classification task.

2. The `fine_tune_bert` function begins by defining custom labels for sentiment analysis tasks. In this case, the labels represent sentiment categories such as "Negative," "Positive," and "Neutral." A mapping is created between these labels and their corresponding numerical IDs (`id2label` and `label2id`). This is essential for ensuring that the model can correctly interpret and output classification labels in the form of human-readable sentiment categories during training and evaluation.
3. A tokenizer is initialized using a pre-trained model, `TinyBERT_General_4L_312D`, from Huawei Noah. The tokenizer is responsible for pre-processing the input text, converting it into a format (tokens) that can be understood by the BERT model. ("A Novel Hybrid Integration of BERT and Conventional Machine Learning ...") This includes padding, truncating, and converting the text into numerical representations (token IDs) that the model can process.
4. A helper function, `tokenize_function`, is used to tokenize the text data. It applies padding and truncation to ensure that each text input is formatted uniformly, with a maximum length of 128 tokens. The tokenized dataset is created by applying this function to the dataset using the `map` method, allowing for batch processing of the data.
5. The `DataCollatorWithPadding` is introduced to dynamically pad the tokenized sequences during training. This ensures that sequences of varying lengths can be processed in batches without issues, as padding makes each sequence within a batch uniform in length.
6. The `TrainingArguments` class is used to specify various training parameters. These include the learning rate, batch sizes, and the number of epochs for training the model. In this case, the learning rate is set to $2e-5$, the model is trained for 3 epochs, and evaluation is performed at the end of each epoch. The model will also save the best version based on the highest accuracy, ensuring optimal performance.
7. The `Trainer` class is instantiated, which simplifies the training loop. The `Trainer` takes the pre-trained model, training arguments, and datasets as inputs, along with the custom metric computation function `compute_metrics`. It manages the entire training and evaluation process, including dynamic padding via the `data_collator`.
8. The `trainer.train()` method initiates the training process, fine-tuning the pre-trained BERT model on the IMDB dataset. Once the training is complete, the model is evaluated on the test set using `trainer.evaluate()`.

The computed evaluation metrics, including accuracy, precision, recall, and F1 score, are printed to the console.

9. After the model has been fine-tuned and evaluated, it is saved locally using the `save_pretrained` method. This saves both the model and the tokenizer, allowing the fine-tuned model to be reused later without retraining. The model's fine-tuning process is then considered complete.

After training of the model, the following results were observed:

Table 3: Model Training and Evaluation Metrics Across Epochs for TinyBERT

Training loss	Epoch	Step	Validation Loss	Precision	Recall	F1	Accuracy
0.3749	3.0000	9375	0.4401	0.8596	0.8594	0.8593	0.8594

Source: Compiled by the author

4.4.11 Annotation

The integration of annotation mechanisms in the text annotation application includes both manual and automatic methods, utilising advanced language models such as TinyBERT and GPT-4 to enhance the annotation process. This section outlines the approaches for manual and semi-automated annotation with TinyBERT and GPT-4 models, emphasising the design and functionalities created to facilitate these tasks.

4.4.11.1 Manual annotation

Manual annotation constitutes the basis of the text annotation process, enabling human annotators to categorise textual material according to established labels or classifications. This is crucial for producing high-quality, gold-standard datasets suitable for training and assessing machine learning models.

4.4.11.2 Manual annotation for named entity recognition

The annotation component is intended to enable manual text annotation inside an intuitive interface. It facilitates the annotation of textual data by allowing users to interactively choose words or phrases and give predefined labels, such as "Person," "Organisation," or other custom categories. This component is crucial for numerous natural language processing (NLP) activities, such as named entity recognition where accurate labelling is vital. The component fundamentally

acquires essential data, including labels and text examples, from the backend via a sequence of asynchronous API calls. It subsequently analyses the raw text, deconstructing it into discrete words and determining their locations. This comprehensive segmentation facilitates precise and intuitive selection during annotating. The component visually depicts these words, emphasising them according to their annotation status and corresponding labels. Users may double-click any word to access a dropdown menu of potential labels, facilitating rapid and efficient annotation. Upon selecting a label, the associated text segment is highlighted, and the annotation is stored both locally and, on the backend, guaranteeing the persistent retention of the annotation data.

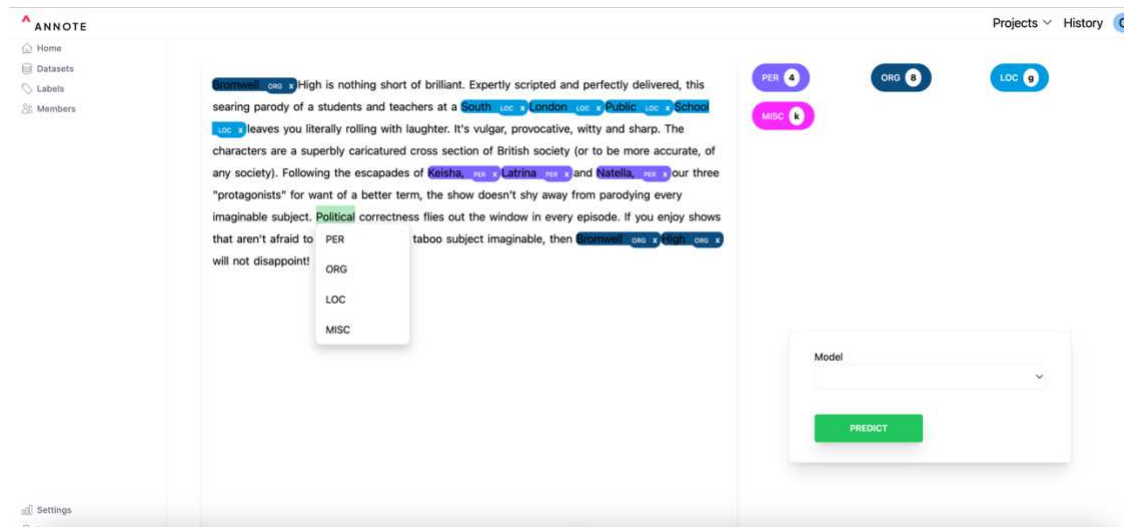


Figure 25: UI for annotating words

4.4.11.3 Manual annotation for sentiment analysis

The classification component is designed to enable users to assign predefined categories or classes to text segments within an intuitive and user-friendly interface. This component is particularly useful for various Natural Language Processing (NLP) tasks such as text classification, topic categorization, and sentiment analysis, where the goal is to categorize text into one or more predefined classes like "Technology," "Business," "Health," or "Positive" and "Negative." It facilitates accurate and efficient labelling of text data, which is essential for training and evaluating machine learning models.

The component comprises two main sections: a "Text Container" and a "Label Selector". The "Text Container" displays the text to be classified, allowing users to

view the content in its entirety. This provides context for accurate classification. The “Label Selector” presents a grid of buttons, each representing a different classification label. When a user selects a label, the corresponding text is marked with that label, providing immediate visual feedback, and allowing users to see the classification status at a glance.

Users can manually select labels by clicking on buttons that represent different categories, such as sentiment or document types. Each label is styled with specific colours for easy identification.

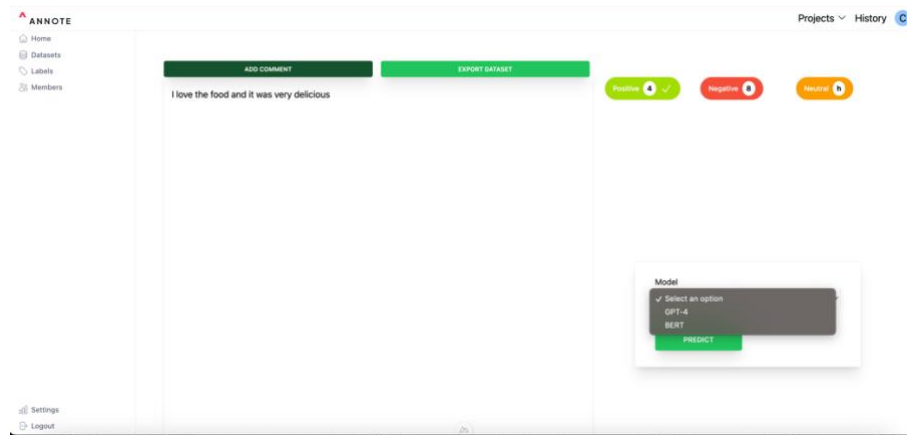


Figure 26: UI for sentiment analysis task

4.4.11.4 Semi-automated annotation with LLMs

Implementing semi-automated annotation for various Natural Language Processing (NLP) tasks such as Named Entity Recognition (NER), sequence classification and sentiment analysis using TinyBERT and GPT-4 involves several key steps, including model fine-tuning, prompt engineering, and integration with APIs. This section details the implementation for leveraging TinyBERT and GPT-4 to automate and enhance the annotation process.

4.4.11.4.1 Named entity recognition with GPT-4

```

1 openai_api_key = settings.OPENAI_API_KEY
2 llm = OpenAI(temperature=0)
3
4 class NER(BaseModel):
5     end_offset:int = Field("An integer representing the position in the text where the named entity ends. It indicates the last character index of the entity")
6     entity_type:str = Field("A string indicating the category or type of the named entity (e.g., 'Person,' 'Location,' 'Organization')")
7     label_name:str = Field("A string representing the actual name or label of the named entity found in the text")
8     start_offset:int = Field("An integer representing the position in the text where the named entity starts. It indicates the first character index of the entity")
9
10
11 class NERList(BaseModel):
12     entities: List[NER]
13
14
15 class docClassification(BaseModel):
16     sentiment:str = Field("A string indicating the sentiment of the text")
17
18 class docClassificationList(BaseModel):
19     inference: List[docClassification]
20
21

```

Figure 27: Data definitions for NER

```

1 def generate_dynamic_prompt(data):
2     entity_types = data['data1']
3     text = data['data2']
4
5     entity_types_str = "\n".join([f"- {entity['text']} : Description for {entity['text']}" for entity in entity_types])
6
7     parser = JsonOutputParser(pydantic_object=NERList)
8
9     prompt_query = (
10         "You are a highly intelligent and accurate Named-entity recognition (NER) system. "
11         "Your task is to recognize and extract specific types of named entities in the given passage. "
12         "Please follow these instructions:\n"
13         "1. Tokenize the text first into words"
14         "2. Only recognize and extract entities that match the following types:\n"
15         f"{entity_types_str}\n"
16         "3. Do not create or use any entity types that are not listed above.\n"
17         "4. If the text does not contain any entities of the specified types, do not include them in the response.\n"
18         "The end_offset and start_offset should be word by word and no overlaps"
19         "Here is the passage:\n"
20         f"{text}\n"
21     )
22
23     prompt = PromptTemplate(
24         template="{prompt_query}{text}\n{format_instructions}",
25         input_variables=["text", "prompt_query"],
26         partial_variables={"format_instructions": parser.get_format_instructions()},
27     )
28
29     chain = prompt | llm | parser
30
31     return chain.invoke({"text": text, "prompt_query": prompt_query})

```

Figure 28: Prompting for NER task

```

1  @csrf_exempt
2  def ner_view(request):
3      if request.method == 'POST':
4          try:
5              data = json.loads(request.body)
6              example_id = data['exampleId']
7              Span.objects.filter(example_id=example_id, annotated_by='llm').delete()
8              if data['selectedModel'] == 'GPT-4':
9                  prompt = generate_dynamic_prompt(data)
10                 print(prompt)
11                 if 'entities' in prompt:
12                     saved_entities = []
13                     required_keys = ['start_offset', 'end_offset']
14                     valid_entities = [entity for entity in prompt['entities'] if all(key in entity for key in required_keys)]
15                     span_types = SpanType.objects.all()
16                     span_type_mapping = {span_type.text: span_type.id for span_type in span_types}
17                     example_id = data['exampleId']
18                     existing_spans = Span.objects.filter(example_id=example_id)
19                     for entity in valid_entities:
20                         entity_type_name = entity['entity_type']
21                         if entity_type_name in span_type_mapping:
22                             start_offset = entity['start_offset']
23                             end_offset = entity['end_offset']
24                             overlaps = any(existing_span.is_overlapping(Span(start_offset=start_offset, end_offset=end_offset, example_id=example_id))
25                                         for existing_span in existing_spans)
26
27                             if overlaps:
28                                 print(f'Skipping overlapping entity: {entity["label_name"]}')
29                                 continue
30                             label_id = span_type_mapping[entity_type_name]
31                             named_entity = Span(
32                                 start_offset=entity['start_offset'],
33                                 end_offset=entity['end_offset'],
34                                 label_id=label_id,
35                                 example_id=data['exampleId'],
36                                 user_id=data['userId'],
37                                 annotated_by='llm'
38                             )
39                             named_entity.save()
40                             saved_entities.append(named_entity)
41                     example_id = data['exampleId']
42                     existing_spans = Span.objects.filter(example_id=example_id)
43                     serializer = SpanSerializer(existing_spans, many=True)
44                     return JsonResponse({'data': serializer.data}, status=200)

```

Figure 29: Backend code for NER

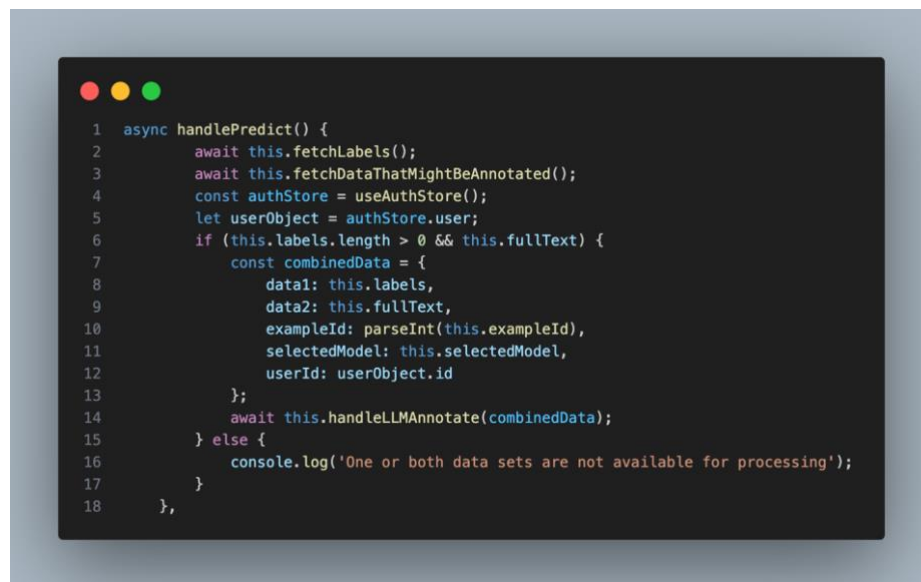
The provided code above is an implementation of a semi-automated Named Entity Recognition (NER) system using the OpenAI API. The system processes a text passage and extracts entities according to predefined categories such as "Person," "Location," or "Organization." This section outlines the key components of the code, detailing how it interacts with the OpenAI model, processes text, and integrates with a Django backend to store and manage the recognized entities.

1. **OpenAI API Initialization:** The OpenAI API key is initialized using the `settings.OPENAI_API_KEY` variable, allowing secure access to the GPT models hosted by OpenAI. The `llm` variable is an instance of the OpenAI model with a specified temperature parameter set to 0. This setting ensures deterministic and consistent responses from the model, which is essential for tasks like NER where reliability is crucial.
2. **Data Structure Definition:** Two classes, `NER` and `NERList`, are defined using the Pydantic library. These classes structure the expected output from the

model, ensuring that the extracted entities conform to a specific schema. Each NER instance includes attributes such as `start_offset`, `end_offset`, `entity_type`, and `label_name`, representing the entity's position and type in the text.

3. **Dynamic Prompt Generation:** The “`generate_dynamic_prompt` function” is responsible for creating a structured prompt for the OpenAI model. It receives a dictionary containing entity types and the text to be processed. The entity types are formatted into a list that specifies which entities the model should recognize and extract. This list is then embedded into a detailed prompt query that guides the model's behavior. The prompt includes clear instructions, such as:
 - i. Tokenizing the text into words.
 - ii. Recognizing only specified entity types.
 - iii. Avoiding overlapping entities.
4. **Prompt Execution:** The prompt is formatted using the `PromptTemplate` class, which combines the input text and prompt query into a single structured input. The template also incorporates formatting instructions from the `JsonOutputParser`, which ensures that the model's output adheres to the predefined structure of the `NERList` class.
5. **Pipeline Execution:** The complete process from prompt generation to parsing the model's response is encapsulated in a processing pipeline defined by the line `chain = prompt | llm | parser`. This line sequentially applies the prompt, sends the text to the OpenAI model, and then parses the output into a structured format.
6. **NER View Function:** The `ner_view` function serves as an API endpoint in the Django application, processing POST requests containing text data and the selected model type.
7. **Prompt Execution and Entity Extraction:** If the selected model is GPT-4, the function generates a dynamic prompt and executes it using the OpenAI model. The returned output is parsed to extract the list of entities recognized by the model.
8. **Entity Validation and Overlap Checking:** Before saving the extracted entities, the function performs several validation checks:
 - i. **Key Presence:** Ensures that each entity contains the required fields such as `start_offset` and `end_offset`.
 - ii. **Type Matching:** Verifies that the entity type matches one of the predefined categories.

- iii. **Overlap Checking:** Checks for overlaps with existing entities in the database to avoid duplications. This is accomplished using the `is_overlapping` method, which compares the start and end offsets of entities.
9. **Entity Storage and Serialization:** Valid entities are then saved to the database as instances of the `Span` model, with relevant metadata such as `example_id`, `label_id`, and `annotated_by`. The function finally serializes the stored entities and returns them as a JSON response to the client.



```
1  async handlePredict() {
2      await this.fetchLabels();
3      await this.fetchDataThatMightBeAnnotated();
4      const authStore = useAuthStore();
5      let userObject = authStore.user;
6      if (this.labels.length > 0 && this.fullText) {
7          const combinedData = {
8              data1: this.labels,
9              data2: this.fullText,
10             exampleId: parseInt(this.exampleId),
11             selectedModel: this.selectedModel,
12             userId: userObject.id
13         };
14         await this.handleLLMAnnotate(combinedData);
15     } else {
16         console.log('One or both data sets are not available for processing');
17     }
18 }
```

Figure 30: Frontend code for automating annotations with LLMs

4.4.11.5 Sentiment analysis with GPT-4

The code below outlines a backend workflow for sentiment analysis using GPT-4, specifically aimed at sentiment analysis. The implementation involves two primary functions: “`classification_prompt`” for generating dynamic prompts and “`docClassification_view`” for handling incoming HTTP requests to classify text data. Below is a comprehensive explanation of how this process works.

1. A Prompt Template is created with specific instructions for the LLM. The prompt template includes:

- i. System Role: Specifies the LLM's role as a "highly intelligent and accurate Sentiment Analysis system."
 - ii. Instructions: The LLM is instructed to classify the sentiment of the provided text based on the given categories.
 - iii. Constraints: The LLM should not create or use any categories not listed and must mark unclear sentiments as 'Unclear'.
 - iv. Text Analysis: The actual text to be analyzed is embedded into the prompt.
2. JsonOutputParser is used to ensure that the LLM's output conforms to the expected docClassificationList format. This parser transforms the LLM's textual output into a structured JSON object that the system can work with programmatically.
 3. The prompt is executed by combining it with the LLM (llm) and the parser, and the result is returned. This function effectively prepares the LLM to provide a structured response for sentiment analysis tasks.
 4. The "docClassification_view" view handles HTTP POST requests for sentiment analysis. It utilizes the classification_prompt function to generate predictions and saves the classified data into the database.

```

1  def classification_prompt(data):
2      category_types = data['data1']
3      text = data['data2']
4
5      category_types_str = "\n".join([f"~ {entity['text']}: Description for {entity['text']}" for entity in category_types])
6
7      parser = JsonOutputParser(pydantic_object=docClassificationList)
8
9      prompt_query = (
10         "You are a highly intelligent and accurate Sentiment Analysis system. "
11         "Your task is to analyze the sentiment expressed in the given passage. "
12         "Please follow these instructions:\n"
13         "1. Identify the overall sentiment of the text: \n"
14         f"{category_types_str}\n"
15         "2. Do not create or use any category types that are not listed above.\n"
16         "3. If the sentiment is mixed or unclear, let the sentiment be 'Unclear'. \n"
17         "Here is the passage:\n"
18         "{text}\n"
19     )
20
21     prompt = PromptTemplate(
22         template="{prompt_query}{text}\n(format_instructions)",
23         input_variables=["text", "prompt_query"],
24         partial_variables={"format_instructions": parser.get_format_instructions()},
25     )
26
27     chain = prompt | llm | parser
28
29     return chain.invoke({"text": text, "prompt_query": prompt_query})

```

Figure 31: Prompt template with GPT-4

```

1  @csrf_exempt
2  def docClassification_view(request):
3      if request.method == 'POST':
4          try:
5              data = json.loads(request.body)
6              example_id = data['exampleId']
7              Category.objects.filter(example_id=example_id).delete()
8              if data['selectedModel'] == 'GPT-4':
9                  prompt = classification_prompt(data)
10                 category_types = CategoryType.objects.all()
11                 category_type_mapping = {category_type.text: category_type.id for category_type in category_types}
12                 sentiment = prompt['inference'][0]['sentiment']
13                 print(sentiment)
14                 sentiment_id = category_type_mapping.get(sentiment)
15                 named_entity = Category(
16                     label_id=sentiment_id,
17                     example_id=data['exampleId'],
18                     user_id=data['userId'],
19                     annotated_by="GPT4-llm"
20                 )
21                 named_entity.save()
22                 example_id = data['exampleId']
23                 existing_category = Category.objects.filter(example_id=example_id)
24                 serializer = CategorySerializer(existing_category, many=True)
25                 return JsonResponse({'data': serializer.data}, status=200)

```

Figure 32: Backend code for sentiment analysis with GPT-4

4.5 Testing

The testing process evaluated the key features of the application to determine how effectively it fulfilled the thesis objectives. The testing included activities such as user registration, project setup, dataset import, annotation workflows, and dataset export, all of which were aligned with the core goals of the application.

The application was designed to deliver a user-friendly interface that minimized complexity and maximized efficiency. Annotators began their tasks by logging into the system, which was straightforward and intuitive. Once logged in, administrators created new annotation projects, defined labels such as named entities or sentiment categories, and added annotators to the projects. Annotators found the project creation and label setup features easy to navigate.

The process of importing datasets into the system was seamless and supported formats, such as text files. Annotators confirmed that the imported datasets were displayed in an organized manner, making it easy to locate specific texts for annotation. The ability to upload and store datasets ensured that the annotation tasks were well-prepared before beginning the actual work.

The annotation interface was another critical aspect of the testing. Annotators worked on datasets by labeling entities or assigning sentiment to texts, either manually or using the assistance of the integrated large language models, GPT-4 and TinyBERT. The interface provided intuitive access to annotation tools and model-generated predictions, which were presented in a structured and visually clear format. Annotators noted the layout for its consistency and ease of use.

Collaboration was a key focus of the testing. Multiple annotators worked on the same dataset to test the system's commenting features. Annotators utilized the commenting feature to provide feedback or clarify ambiguous annotations, which enhanced communication and teamwork. Role-based permissions effectively controlled access to specific projects and tasks, allowing administrators to manage the workflow efficiently. These collaborative features were crucial in ensuring that the annotation process was cohesive and productive.

To ensure the quality of annotations, the application was assessed by comparing the outputs against established gold-standard datasets, such as the Twitter Sentiment Analysis dataset for sentiment analysis and a Kaggle NER dataset for named entity recognition.

The semi-automation features integrated into the application were particularly valuable in reducing the manual workload. Annotators performed tasks both with and without automation to evaluate the impact of the large language models on efficiency. Automation reduced the average annotation time with GPT-4 excelling in zero-shot scenarios and TinyBERT providing precise token-level suggestions for structured datasets. Annotators expressed satisfaction with the automation tools, noting how they streamlined repetitive tasks and improved productivity. By combining automated suggestions with manual refinement, the application struck a balance between efficiency and accuracy, ensuring that high-quality annotations were consistently produced.

4.5.1 Sentiment analysis with GPT-4 and BERT

A comparison of GPT-4 and the fine-tuned BERT model for the sentiment analysis task is presented in the table below. The dataset for sentiment analysis comprised 300 samples, with 50% labelled as positive and 50% as negative. The metrics were calculated using the scikit-learn library's `precision_recall_f1` `score_support` and `accuracy_score` functions, applied to both models' predictions and the gold standard labels.

Below is a table illustrating the true labels and the predictions made by GPT-4 and BERT for some of the samples:

Twitter Post	True Label	GPT-4 Prediction	BERT Prediction
Thank you for hanging up everyone! It was fun. I'll try to get a haircut tomorrow, but if it wasn't for me, some IRL streams would still be in Hawaii. We'll see. Thank you so much for the @ mompou _ mumpow @ MegaMagwife and @ fendMitch raids	Positive	Positive	Positive
Borderlands 2 is so good omg	Positive	Positive	Positive
What a night! I had heaps of fun playing Borderlands 2! We're getting closer and closer towards the end! . . After the stream we raided @ClocksyDraws ! What a amazing streamer with beautiful art pieces! Give him a follow if you don't already do! . . Goodnight!.	Positive	Positive	Positive
Y0000000000 i'm currently so hyped for this movie	Positive	Positive	Positive
All these games are good this year and are the best of their kind.	Positive	Positive	Positive
The next one is Borderlands: Best Handsome Collection. You're welcome	Positive	Positive	Positive
The next short one is Borderlands : For The True Handsome Collection. You're welcome	Positive	Positive	Positive

The second one is Borderlands: The Handsome Collection. How're welcome	Positive	Positive	Positive
This April patch + mini DLC for Borderlands 3 is amazing. The events, the new difficulty modifiers, the ramp up in gear stats, all combine to make the game fresh yet again.	Positive	Positive	Positive
Positive,Big night hearty congratulations for to having my @PlayApex campaign @Respawn project @CallofDuty @Borderlands @DestinyTheGame Distinguished families putting on your numerous @BAFTAGames Foundation nominations!!! Woohooooo!!!!	Positive	Positive	Positive
This is bad. I get that profit-wise it was less than expected due to a huge budget.	Negative	Negative	Negative
This is crappy. I get that it was less profit than expected due to a huge budget.	Negative	Negative	Negative
frustrated. pic.twitter.com/Wav1bacr5j	Negative	Negative	Negative
""""What a bad woman!""""	Negative	Negative	Negative
All the Borderlands are rubbish	Negative	Negative	Negative
I hate that this easy horrible modifier event on game 10 last forever.	Negative	Negative	Negative

Table 4: Sentiment Analysis Results for GPT-4 and BERT on Sample Tweets

4.5.2 Named entity recognition with GPT-4 and TinyBERT

The accuracy, precision, recall and f1-score were calculated for each document and then the average was obtained for each of the metrics.

Table 5 displays predictions produced by GPT-4 and TinyBERT for one of the documents:

Table 5: Named Entity Recognition Results for TinyBERT and GPT-4 on a document

Doc ID	Document	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Doc 1	Energy, security, climate change, the World Trade Organization's stalled negotiations, and investment are all expected to be among the major topics in the annual report of the APEC Business Advisory Council , known as ABAC	TinyBERT	100	100	100	100
	Energy, security, climate change, the World Trade Organization 's stalled negotiations, and investment are all expected to be among the major topics in the annual report of the APEC Business Advisory Council , known as ABAC	GPT-4	88.89	50	50	50

Source: Compiled by author

5 Discussion

This section presents a review of testing the application by working on text annotation tasks. The text annotation system was designed with a strong focus on user-friendliness, collaboration, and automation. Key features include a role-based user management system, an intuitive annotation interface, integration of semi-automation for annotations, a model fine-tuning interface, collaborative tools for feedback, and flexible data export options. Each of these components plays a crucial role in streamlining workflows, supporting quality assurance, and reducing manual workload, ultimately making the system highly functional for both technical and non-technical users.

The system was designed with a strong emphasis on user-friendliness, incorporating an intuitive annotation interface that supports both manual and semi-automated annotations. The evaluation process included tasks such as highlighting text, applying labels, and reviewing model-suggested annotations. Annotators reported that the interface was easy to navigate, with features such as consistent tool placement and responsive design contributing to a positive user experience.

To further validate the usability of the interface, the system was compared to other widely used annotation tools, such as MMAX2 and WordFreak. While these tools provide robust functionality, users noted that the inclusion of features like integrated model suggestions and customizable annotation workflows in our system reduced cognitive load and simplified the annotation process.

The integration of large language models (GPT-4, BERT, and TinyBERT) enables semi-automated annotations, reducing manual workload while maintaining high quality. Annotators performed tasks with and without automation to assess its impact, with automation reducing annotation time. For example, GPT-4 provided high-quality zero-shot predictions for sentiment analysis, while TinyBERT excelled in structured named entity recognition tasks.

To evaluate annotation quality, outputs were compared against gold-standard datasets. This feature, absent in BRAT and Textinator, provided a significant advantage for projects requiring precision.

The system provides flexible data import and export options, supporting formats such as JSONL. During testing, annotators successfully imported datasets without encountering significant errors, thanks to the integrated validation mechanisms.

Annotated datasets were exported seamlessly, enabling further analysis in external tools.

In comparison, Prodigy and Label Studio also offer flexible data handling but lack the same level of error feedback for unsupported file formats. Annotators appreciated the system's clear error messages and guidance, which minimized disruptions during the workflow.

The system was tested for its ability to support collaboration among multiple users. Three annotators worked on the same project simultaneously, testing features such as role-based permissions and the "Comments" feature. The role-based access control system ensured that annotators, reviewers, and administrators could focus on their specific tasks without interference, enabling structured workflows. The "Comments" feature allowed annotators to provide contextual feedback and resolve ambiguities collaboratively, which was particularly useful during the review process.

5.1 Limitations of the application

Although large language models (LLMs) like GPT-4, BERT, and TinyBERT have formidable capabilities for text annotation tasks, including sentiment analysis and named entity recognition (NER), some constraints hinder their performance and applicability in practical scenarios. The limits encompass intrinsic model design issues, computing constraints, generalisation difficulties, and perhaps ethical concerns.

1. **Hallucinations in GPT-4:** A unique challenge posed by GPT-4 is the tendency to generate hallucinations plausible-sounding but incorrect or fabricated information. In text annotation tasks, hallucinations manifested as the incorrect identification of entities or sentiment did not actually exist in the text. For example, in NER, GPT-4 labelled an entity in the text that is not present without supporting evidence in the document. This hallucination problem reduced the reliability of GPT-4 in tasks where accuracy and factual consistency are critical. While fine-tuning helps reduce hallucinations, they remain a notable limitation of generative models like GPT-4 (Huang et al., 2023).
2. **Computational and resource constraints:** The computational demands of deploying LLMs were another major limitation, especially when training the BERT model. LLMs like GPT-4 as well require substantial GPU or TPU resources for both training and inference. In text annotation systems that require real-time performance or are deployed in low-resource

environments, the high computational cost may be prohibitive. These resource constraints also limited the ability to fine-tune models regularly on new data, which may be necessary to maintain their accuracy in changing domains (Bai et al., 2024).

3. Contextual understanding: Although LLMs excel at processing and generating text, they struggled with nuanced contextual understanding, particularly in highly specialized or technical domains. In tasks like NER, GPT-4 misinterpreted specialized terminology or missed key entities. This limitation led to inaccurate or incomplete annotations (Zhu et al., 2024).
4. Handling long-form or complex texts: Another limitation was GPT-4's difficulty in processing long-form or complex documents. GPT-4 has a token limit, and when dealing with lengthy legal documents, academic papers, or reports, it may struggle to maintain consistency or accurately capture entities throughout the document. This limitation forces users to break the text into smaller chunks for processing, which can lead to a loss of context and degraded performance in tasks like entity recognition or sentiment analysis (Du et al., 2023).
5. Limitation to hugging face datasets for fine-tuning: The text annotation system's fine-tuning functionality is restricted to datasets available on Hugging Face. While this allows for seamless integration with a variety of established datasets, it limits users who may need to fine-tune models on custom or proprietary datasets. This constraint impacts the system's flexibility, particularly for organizations working with domain-specific data that may not be publicly available on Hugging Face. As a result, users are restricted to fine-tuning on pre-existing datasets, potentially limiting the system's effectiveness in specialized contexts.

6 Conclusion

This thesis reveals the intricate architecture and functionality of the text annotation system, emphasizing its capacity to optimize operations and improve annotation quality. The system's key characteristics, such as an intuitive user interface, automation capabilities, collaborative tools, and flexible data export options, render it adaptable for various annotation activities and user requirements. The system employs modern models such as GPT-4, BERT, and TinyBERT to significantly reduce manual labour while ensuring high accuracy and reliability.

The application features a customizable interface, which allows users to tailor the system to their specific annotation needs. For example, administrators can define annotation labels. Users also have control over the annotation environment, such as adjusting settings for semi-automation (e.g., selecting between manual or model-assisted annotations) and defining fine-tuning parameters for models. This flexibility ensures that the system can accommodate diverse datasets and annotation scenarios, making it suitable for both technical and non-technical users.

7 References

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019, June). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 4171–4186. Minneapolis, MN: Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>. Available at: <https://aclanthology.org/N19-1423>.

Törnberg, P. (2023). ChatGPT-4 outperforms experts and crowd workers in annotating political Twitter messages with zero-shot learning. arXiv preprint, arXiv:2304.06588. <https://arxiv.org/abs/2304.06588>

Mitkov, R. (2003). The Oxford Handbook of Computational Linguistics. Oxford: Oxford University Press, ISBN 978-0-19-927634-9.

Goldberg, Y. (2017). Neural network methods for natural language processing. Morgan & Claypool. <https://doi.org/10.2200/S00762ED1V01Y201703HLT037>

Alizadeh, M., Kubli, M., Samei, Z., Dehghani, S., Zahedivafa, M., Bermeo, J. D., Korobeynikova, M., & Gilardi, F. (2023). Open-source LLMs for text annotation: A practical guide for model setting and fine-tuning. University of Zurich.

Heseltine, M., Clemm Von Hohenberg, B. Large Language Models as a Substitute for Human Experts in Annotating Political Text. 2023. <https://doi.org/10.31219/osf.io/cx752>.

Jurafsky, D., Martin, J. H. Speech, and Language Processing. 3. vyd., draft. 2020. <https://web.stanford.edu/~jurafsky/slp3/>.

Stenetorp, P., Pyysalo, S., Topić, G., Ohta, T., Ananiadou, S., & Tsujii, J. (2012). brat: A web-based tool for NLP-assisted text annotation. In F. Segond (Ed.), Proceedings of the Demonstrations at the 13th Conference of the European Chapter of the Association for Computational Linguistics (pp. 102–107). Association for Computational Linguistics. <https://aclanthology.org/E12-2021>

OpenAI. (2023). GPT-4 Technical Report. Retrieved from <https://cdn.openai.com/papers/gpt-4.pdf>.

Nuxt3, 2024a. Introduction. Introduction | Nuxt3 [online] [cit. 14 April 2024]. Available at: <https://nuxt.com/docs/getting-started/introduction>

Patil, Chandan. (2022). NLP Assisted Text Annotation. International Journal of Scientific Research in Engineering and Management. 06. 10.55041/ijrsrem14651.

PostgreSQL, 2024. Introduction. Introduction | PostgreSQL [online] [cit. 14 April 2024]. Available at: <https://www.postgresql.org/about/>

Django, 4.0| Django [online] [cit. 14 April 2024]. Available at: <https://docs.djangoproject.com/en/4.0/>

Redis, 5.0. Available at: <https://redis.io/>

Sommerville, I. (2011) Software Engineering. 9th Edition, Pearson.

Activity Diagram | [online] [cit. 8 June 2024]. Available at: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/>

Vaswani, A., Shazeer, N.M., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., & Polosukhin, I. (2017). Attention is All you Need. Neural Information Processing Systems.

Nyandwi, Jean. 2023. "The Transformer Blueprint: A Holistic Guide to the Transformer Neural Network Architecture." Deep Learning Revision, July. <https://deepprevious.github.io/posts/001-transformer/>.

Jiao, X., Yin, Y., Shang, L., Jiang, X., Chen, X., Li, L., Wang, F., & Liu, Q. (2020). TinyBERT: Distilling BERT for natural language understanding. arXiv. <https://arxiv.org/abs/1909.10351>

White J et al (2023) A prompt pattern catalog to enhance prompt engineering with ChatGPT.arXiv cs.SE[20] White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C (2023). A prompt pattern catalog to enhance prompt engineering with chatgpt. arXiv preprint arXiv:2302.11382.

Erik F. Tjong Kim Sang and Fien De Meulder. 2003. Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition. In Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003, pages 142–147.

Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. In Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies (pp. 142-150). Association for Computational Linguistics.
<https://www.aclweb.org/anthology/P11-1015>

Alqaydeh, N. (2021). Named Entity Recognition (NER) Corpus [Data set]. Kaggle.
<https://www.kaggle.com/datasets/naseralqaydeh/named-entity-recognition-ner-corpus>

JP797498e. (2021). Twitter Entity Sentiment Analysis [Data set]. Kaggle.
<https://www.kaggle.com/datasets/jp797498e/twitter-entity-sentiment-analysis>

BlueberryML. (n.d.). Image of large language models. BlueberryML.
<https://blueberryml.com/llm>

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://doi.org/10.48550/arXiv.2005.14165>

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023). *Llama 2: Open foundation and fine-tuned chat models*. arXiv preprint arXiv:2307.09288.

Wang, X., Kim, H., Rahman, S., Mitra, K., & Miao, Z. (2024). Human-LLM collaborative annotation through effective verification of LLM labels. In Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (Article 303, pp. 1–21). Association for Computing Machinery.
<https://doi.org/10.1145/3613904.3641960>

Yu, Y., Li, J., Liu, H., Yang, F., & Yu, X. (2024). Text classification system based on LLM. *Journal of Artificial Intelligence Practice*, 7(1), 172-178.
<https://doi.org/10.23977/jaip.2024.070321>

Keraghel, I., Morbieu, S., & Nadif, M. (2024). A survey on recent advances in named entity recognition. Centre Borelli UMR9010, Université Paris Cité.

Sebastiani, F. (2001). Machine learning in automated text categorization. Consiglio Nazionale delle Ricerche, Italy. Retrieved from <http://arxiv.org/abs/cs/0110053v1>.

Leech, G. (1997). Corpus annotation: Linguistic information from computer text corpora, chapter Introducing corpus annotation, pages 1–18. Longman, Londres, England.

Song, M., Feng, Y., & Jing, L. (2023). A survey on recent advances in keyphrase extraction from pre-trained language models. In A. Vlachos & I. Augenstein (Eds.), Findings of the Association for Computational Linguistics: EACL 2023 (pp. 2153–2164). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.findings-eacl.161>

Singh, J., Joshi, N., & Mathur, I. (2013). Part of speech tagging of Marathi text using trigram method. International Journal of Advanced Information Technology, 3(2), 35–41. <https://doi.org/10.5121/ijait.2013.3203>

Pernelle, N. (2016). Semantic enrichment of data: Annotation and data linking (Habilitation à Diriger des Recherches). Université Paris-Sud, Orsay, France.

Neves M, Ševa J. An extensive review of tools for manual annotation of documents. Brief Bioinform. 2021 Jan 1;22(1):146–63. doi: 10.1093/bib/bbz130.

Bonet-Jover, A., Sepúlveda-Torres, R., Saquete, E., & Martínez-Barco, P. (2023). A semi-automatic annotation methodology that combines summarization and human-in-the-loop to create disinformation detection resources. Knowledge-Based Systems, 275, 110723. <https://doi.org/10.1016/j.knosys.2023.110723>

Finin, T., Murnane, W., Karandikar, A., Keller, N., Martineau, J., & Dredze, M. (Year). Annotating named entities in Twitter data with crowdsourcing. (2010)

Al-Laith, A., Shahbaz, M., Alaskar, H., & Rehmat, A. (2021). AraSenCorpus: A semi-supervised approach for sentiment annotation of a large Arabic text corpus. Applied Sciences, 11(5), 2434. <https://doi.org/10.3390/app11052434>

Ding, B., Chengwei, Q., Linlin, L., Yew, C., Shafiq J., Boyang, L., and Lidong B. (2023). Is GPT-3 a Good Data Annotator

Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., & Mian, A. (2024). A comprehensive overview of large language models
 [41] Wang, C., Li, M., & Smola, A. J. (2019). Language models with transformers.

Raiaan, M. A. K., Mukta, M. S. H., Fatema, K., Fahad, N. M., Sakib, S., Mim, M. M. J., Ahmad, J., Ali, M. E., & Azam, S. (2024). A review on large language models: Architectures, applications, taxonomies, open issues and challenges. IEEE Access, 12, 26839-26874. <https://doi.org/10.1109/ACCESS.2024.3365742>

Morton, T., & LaCivita, J. (2003). WordFreak: An open tool for linguistic annotation. In Companion Volume of the Proceedings of HLT-NAACL 2003 - Demonstrations (pp. 17-18). Association for Computational Linguistics. <https://aclanthology.org/N03-4009/>

Müller, C., & Strube, M. (2006). Multi-Level Annotation of Linguistic Data with MMAX2. <https://mmax2.net/mmaxpaper.pdf>

Nakayama, H., Kubo, T., Kamura, J., Taniguchi, Y., & Liang, X. (2018). doccano: Text annotation tool for human. Software available from <https://github.com/doccano/doccano>.

Explosion AI. (n.d.). Prodigy: An annotation tool for machine learning and NLP. Retrieved from <https://prodigy/>

HumanSignal. (n.d.). Label Studio: Open-source data labeling tool. Retrieved from <https://labelstud.io/>

Kalpakchi, D., & Boye, J. (2022). Textinator: An internationalized tool for annotation and human evaluation in natural language processing and generation. In N. Calzolari, F. Béchet, P. Blache, K. Choukri, C. Cieri, T. Declerck, S. Goggi, H. Isahara, B. Maegaard, J. Mariani, H. Mazo, J. Odijk, & S. Piperidis (Eds.), Proceedings of the Thirteenth Language Resources and Evaluation Conference (pp. 856-866). European Language Resources Association. <https://aclanthology.org/2022.lrec-1.90>

Google. (2024). Angular: Developer Guide. Retrieved from <https://angular.dev/overview>

You, E., & the Vue.js team. (2024). Vue.js: The Progressive JavaScript Framework. Retrieved from <https://vuejs.org>

[51] Facebook. (2024). React: A JavaScript Library for Building User Interfaces. Retrieved from <https://reactjs.org>

Ronacher, A. (2010). Flask: Web Development, One Drop at a Time. Retrieved from <https://flask.palletsprojects.com/>

Dahl, R. (2009). Node.js: JavaScript on the Server Side. Retrieved from <https://nodejs.org/en/about/>

Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems. Pearson.

Oracle Corporation. (2023). MySQL Documentation. Retrieved from <https://dev.mysql.com/doc/>

Nielsen, J. (1993). Usability Engineering. Morgan Kaufmann.

Fort, K., Adda, G., & Cohen, K. B. (2011). Last words: Amazon Mechanical Turk: Gold mine or coal mine? Computational Linguistics, 37(2), 413–420. https://doi.org/10.1162/COLI_a_00057

Shneiderman, B., & Plaisant, C. (2004). Designing the User Interface: Strategies for Effective Human-Computer Interaction (4th ed.). Pearson.

Bass, L., Clements, P., & Kazman, R. (2003). Software Architecture in Practice (2nd ed.). Addison-Wesley.

Ribeiro, R. (2022). Vue.js 3 Cookbook: Building User-Centric Frontends with Nuxt 3. Packt Publishing.

Sharma, Aneeta. (2018). Hands-On Full-Stack Development with Vue.js and Node. Packt Publishing.

Holovaty, A., & Kaplan-Moss, J. (2009). The Definitive Guide to Django: Web Development Done Right. Apress.

Forcier, P., Bissex, B., & Chun, W. (2009). Python Web Development with Django. Addison-Wesley.

Momjian, B. (2001). PostgreSQL: Introduction and Concepts. Addison-Wesley.

Kovář, V., Jakubíček, M., & Horák, A. (2016). On evaluation of natural language processing tasks: Is gold standard evaluation methodology a good solution? Natural Language Processing Centre, Faculty of Informatics, Masaryk University, Brno.

Powers, D. M. W. (2011). Evaluation: From Precision, Recall, and F-Measure to ROC, Informedness, Markedness & Correlation. *Journal of Machine Learning Technologies*, 2(1), 37–63.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2023). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv*. <https://arxiv.org/abs/2311.05232>

Bai, G., Chai, Z., Ling, C., Wang, S., Lu, J., Zhang, N., Shi, T., Yu, Z., Zhu, M., Zhang, Y., Yang, C., Cheng, Y., & Zhao, L. (2024). Beyond efficiency: A systematic survey of resource-efficient large language models. *Journal*.

Zhu, Y., Moniz, J. R. A., Bhargava, S., Lu, J., Piraviperumal, D., Li, S., Zhang, Y., Yu, H., & Tseng, B.-H. (2024). Can large language models understand context?

Du, Z., Jiao, W., Wang, L., Lyu, C., Pang, J., Cui, L., Song, K., Wong, D., Shi, S., & Tu, Z. (2023). On extrapolation of long-text translation with large language models. <https://doi.org/10.13140/RG.2.2.11613.59361>

Bharathi Mohan, G., Prasanna Kumar, R., Vishal Krishh, P. et al. An analysis of large language models: their impact and potential applications. *Knowl Inf Syst* 66, 5047–5070 (2024). <https://doi.org/10.1007/s10115-024-02120-8>

Asher, N., Benamara, F., & Mathieu, Y. Y. (2008). Distilling opinion in discourse: A preliminary study. In *Proceedings of the International Conference on Computational Linguistics (COLING-2008): Companion Volume: Posters and Demonstrations*.

Weld, H., Huang, X., Long, S., Poon, J., & Han, S. C. (2021). A survey of joint intent detection and slot-filling models in natural language understanding. *The University of Sydney*.

Kalra, V., Kashyap, I. & Kaur, H. Improving document classification using domain-specific vocabulary: hybridization of deep learning approach with TFIDF. *Int. j. inf. technol.* 14, 2451–2457 (2022). <https://doi.org/10.1007/s41870-022-00889-x>

Ratner, A., Bach, S., Ehrenberg, H., Fries, J., Wu, S., & Ré, C. (2020). Snorkel: Rapid training data creation with weak supervision. *The VLDB Journal*, 29, 709–730. <https://doi.org/10.1007/s00778-019-00552-1>

Zhang, W., Deng, Y., Liu, B., Pan, S. J., & Bing, L. (2023). Sentiment Analysis in the Era of Large Language Models: A Reality Check. Proceedings of the 2023 Association for Computational Linguistics (ACL).

Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval*, 2(1-2), 1-135.
<https://doi.org/10.1561/15000000011>

8 Abbreviations

API	Application programming interface
REST	Representational State Transfer
BERT	Bidirectional Encoder Representations from Transformers
HTML	The HyperText Markup Language
HTTP	Hypertext Transfer Protocol
NLP	Natural language processing
LLM	Large Language Model
GPT	Generative Pre-Trained Transformer
ML	Machine Learning
NER	Named Entity Recognition
POS	Part of Speech
SSR	Server-Side Rendering
SSG	Static Site Generation
CSR	Client-Side Rendering
SEO	Search Engine Optimization
DRF	Django REST Framework
AI	Artificial Intelligence
RDMS	Relational Database Management System
SQL	Structured Query Language
JSON	Javascript Object Notation
JSONB	Javascript Object Notation as Binary Representation
JSONL	Javascript Object Notation Lines
DRF	Django Rest Framework
UML	Unified Modeling Language
GLUE	General Language Understanding Evaluation

9 List of figures

Figure 1: Transformer architecture. Adapted from BlueberryML	21
Figure 2: Use case diagram for text annotation application.....	44
Figure 3: Figma design for sentiment analysis task	46
Figure 4: Figma design for commenting on an annotation task.....	47
Figure 5: Figma design for NER task.....	47
Figure 6: Activity diagram for annotating text	50
Figure 7: Code for defining connection to database	52
Figure 9: Backend code for processing registration request.....	53
Figure 10: UI for registering user	53
Figure 11: UI for creating a project.....	54
Figure 12: UI for creating a label.....	55
Figure 13: UI for uploading text file.....	57
Figure 14: Frontend for adding an annotator	58
Figure 15: Frontend for comments functionality	59
Figure 16: Frontend for fine-tuning.....	60
Figure 17: Backend code for importing modules	60
Figure 18: Backend code for loading dataset.....	61
Figure 19: features for CoNLL 2003 dataset.....	61
Figure 20: Backend code for finetuning model.....	62
Figure 21: Backend code for calculating metrics.....	64
Figure 22: Code for finetuning and publishing model	64
Figure 23: Backend code for importing modules for finetuning.....	66
Figure 24: Backend code for finetuning model.....	67
Figure 25: Code for finetuning BERT.....	68
Figure 26: UI for annotating words	71
Figure 27: UI for sentiment analysis task.....	72
Figure 28: Data definitions for NER.....	73
Figure 29: Prompting for NER task.....	73
Figure 30: Backend code for NER	74
Figure 31: Frontend code for automating annotations with LLMs.....	76
Figure 33: Prompt template with GPT-4	77
Figure 34: Backend code for sentiment analysis with GPT-4.....	78

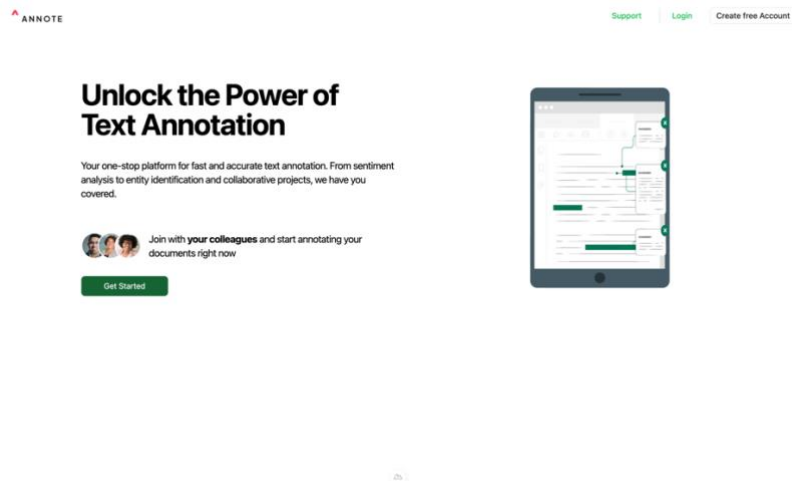
11 List of tables

Table 1: Comparison of text annotation applications based on key criteria	30
Table 2: Model Training and Evaluation Metrics Across Epochs for BERT	66
Table 3: Model Training and Evaluation Metrics Across Epochs for TinyBERT ...	70
Table 4: Sentiment Analysis Results for GPT-4 and BERT on Sample Tweets	82
Table 5: Named Entity Recognition Results for TinyBERT and GPT-4 on a document	83

Attachments

Link to code on Github - <https://github.com/gaby96/TextAnnotationThesis>

The attachment section contains screenshots of some components of the UI to offer a comprehensive view of certain aspects that were not fully highlighted in the main body of the thesis. These screenshots aim to provide a more in-depth understanding of the application's interface, functionality, and design elements, which support the concepts and methodologies discussed in previous sections.



ANNOTE

Home

Datasets

Labels

Members

Settings

Logout

Projects

History

Import Dataset

Status	Text	Action
In progress	Busy Phillips ...	
In progress	im getting on ...	
In progress	im coming on ...	
In progress	So I spent a c...	
In progress	2010 So I spe...	
In progress	that was the fi...	
In progress	WE FINISHED...	
In progress	Thank you for...	
In progress	I am coming L...	
In progress	im getting on ...	

items per page: 25 1-10 of 10

ANNOTE

Home

Settings

Logout

Projects

History

Add Project

Named Entity Recogniton

Description
named entity task

Project Type
SequenceLabeling

Author
pmrossan@gmail.com

Tags

Project Doc Classification

Description
Project Desc

Project Type
DocumentClassification

Author
pmrossan@gmail.com

Tags
classification document

Document Classification Project

Description
This is a doc classification project

Project Type
DocumentClassification

Author
pmrossan@gmail.com

Tags

Sequence Labeling Project

Description
This is a description for sequence labeling

Project Type
SequenceLabeling

Author
pmrossan@gmail.com

Tags
entity sequence